



《模式识别》

第五章 EM算法与聚类

马锦华

<https://cse.sysu.edu.cn/teacher/MaJinhua>

SUN YAT-SEN University



声明：该PPT只供非商业使用，也不可视为任何出版物。由于历史原因，许多图片尚没有标注出处，如果你知道图片的出处，欢迎告诉我们 majh8@mail.sysu.edu.cn.



课程目录（暂定）

❑	第一章	课程简介与预备知识	6学时
❑	第二章	特征提取与表示	6学时
❑	第三章	主成分分析	3学时
❑	第四章	归一化、判别分析、人脸识别	3学时
❑	第五章	EM算法与聚类	3学时
❑	第六章	贝叶斯决策理论	3学时
❑	第七章	线性分类器与感知机	3学时
❑	第八章	支持向量机	3学时
❑	第九章	神经网络、正则项和优化方法	3学时
❑	第十章	卷积神经网络及经典框架	3学时
❑	第十一章	循环神经网络	3学时
❑	第十二章	Transformer	3学时
❑	第十三章	自监督与半监督学习	3学时
❑	第十四章	开放世界模式识别	6学时



What we will learn today?

- Introduction to segmentation and clustering
- Agglomerative clustering
- K-means clustering
- Mean-shift clustering
- Expectation-Maximization (EM)

Image Segmentation

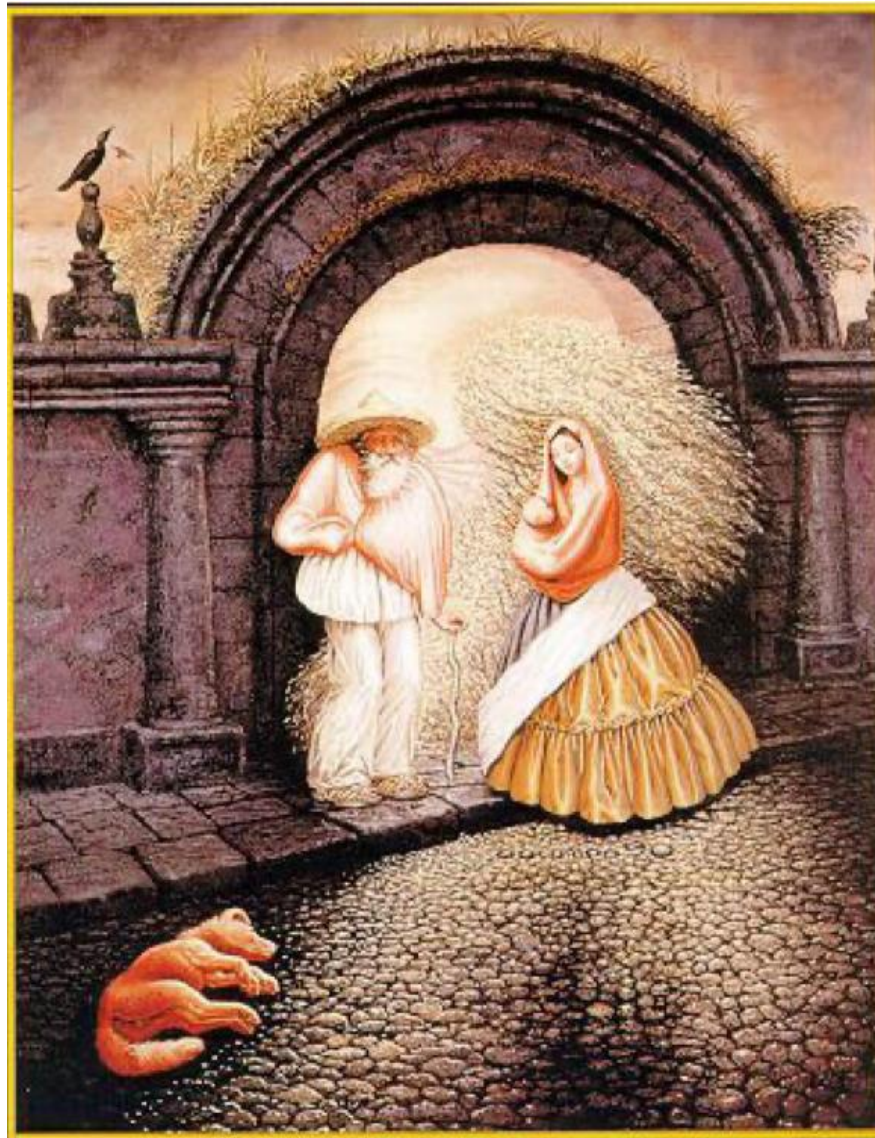
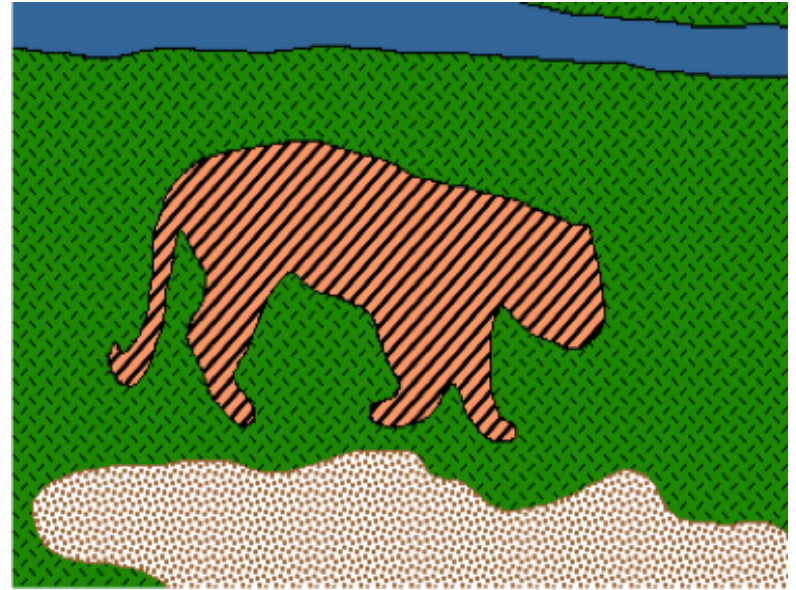


Image Segmentation

- Goal: identify groups of pixels that go together



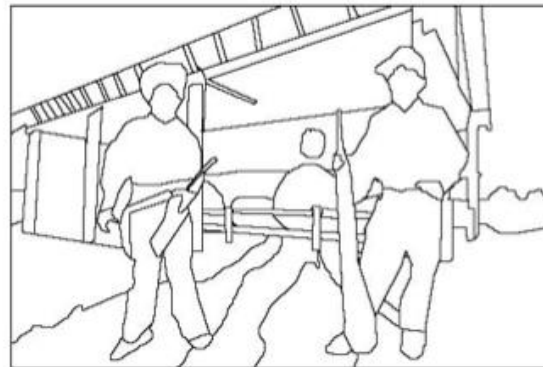
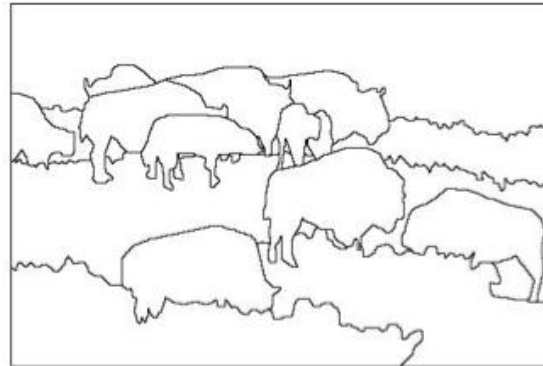
The Goals of Segmentation

- Separate image into coherent “objects”

Image



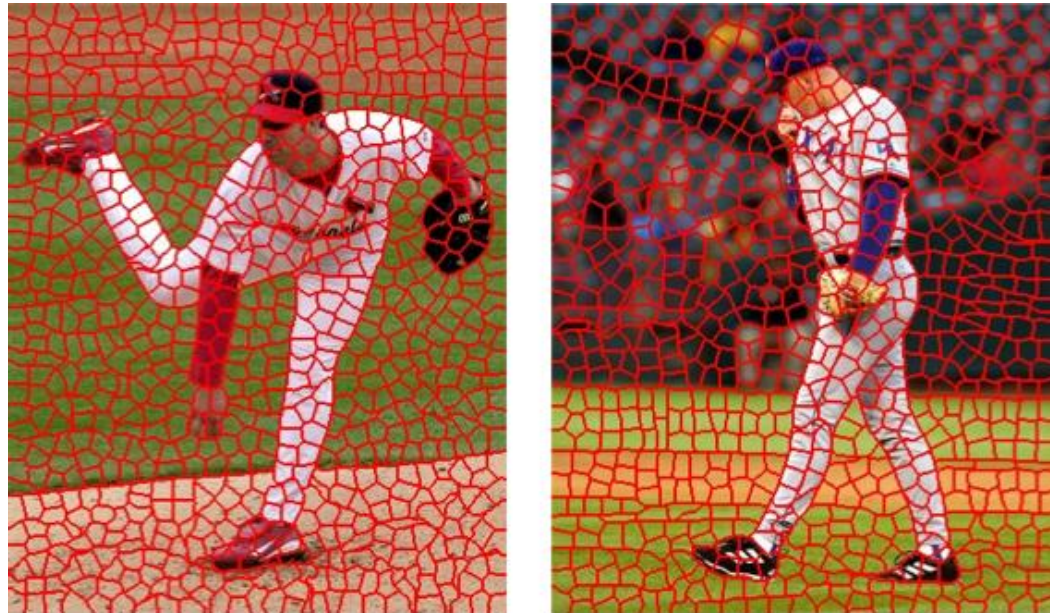
Human segmentation



The Goals of Segmentation

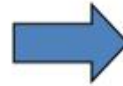
- Separate image into coherent “objects”
- Group together similar-looking pixels for efficiency of further processing

“superpixels”

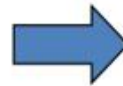


X. Ren and J. Malik. Learning a classification model for segmentation. ICCV 2003.

Segmentation for efficiency



[Felzenszwalb and Huttenlocher 2004]



[Hoiem et al. 2005, Mori 2005]

[Shi and Malik 2001]

Segmentation as a result

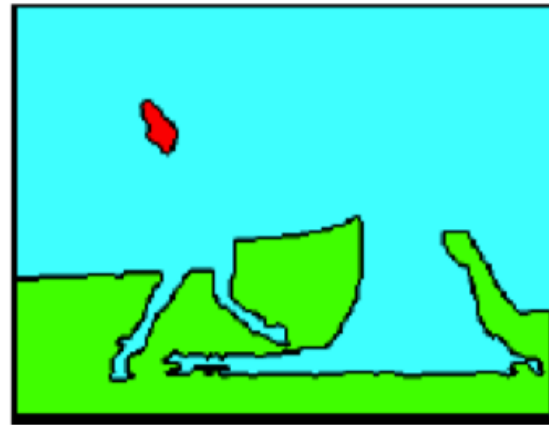


Rother et al. 2004

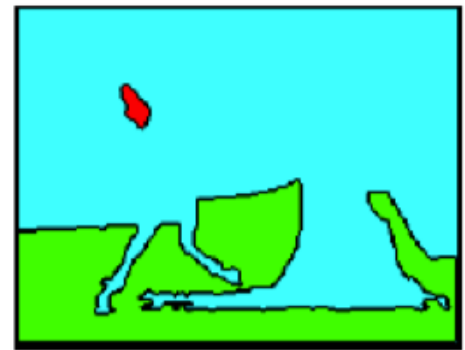
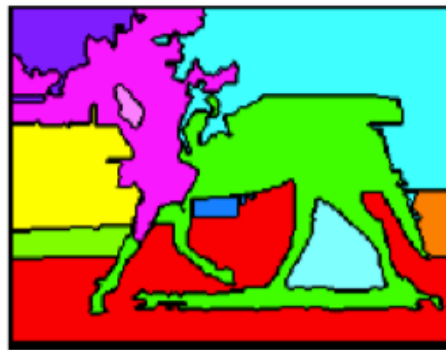
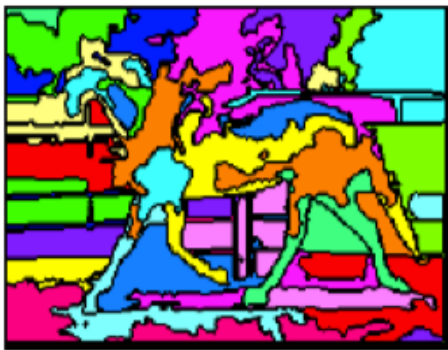
Types of segmentations



Oversegmentation



Undersegmentation



Multiple Segmentations



One way to think about “segmentation” is Clustering

Clustering: group together similar data points and represent them with a single token

Key Challenges:

- 1) What makes two points/images/patches similar?
- 2) How do we compute an overall grouping from pairwise similarities?

Why do we cluster?

- **Summarizing data**

- Look at large amounts of data
- Patch-based compression or denoising
- Represent a large continuous vector with the cluster number

- **Counting**

- Histograms of texture, color, SIFT vectors

- **Segmentation**

- Separate the image into different regions

- **Prediction**

- Images in the same cluster may have the same labels

How do we cluster?

- **Agglomerative clustering**

- Start with each point as its own cluster and iteratively merge the closest clusters

- **K-means**

- Iteratively re-assign points to the nearest cluster center

- **Mean-shift clustering**

- Estimate modes of pdf

General ideas

- **Tokens**
 - whatever we need to group (pixels, points, surface elements, etc., etc.)
- **Bottom up clustering (e.g. agglomerative clustering)**
 - tokens belong together because they are locally coherent
- **Top down clustering (e.g. divisive clustering)**
 - tokens belong together because they lie on the same visual entity (object, scene...)

Notes: These two are not mutually exclusive

Examples of Grouping in Vision



Determining image regions



Grouping video frames into shots

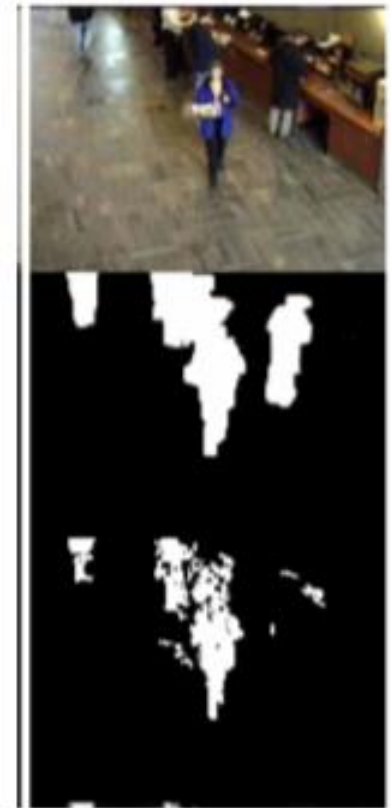
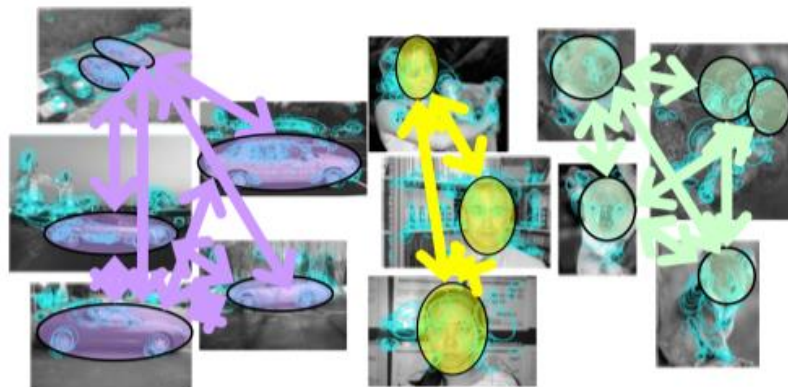


Figure-ground

*What things should
be grouped?*

*What cues
indicate groups?*

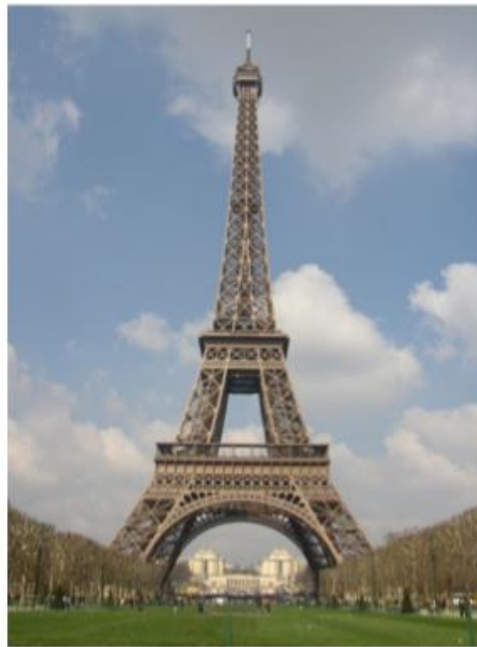


Object-level grouping

Similarity



Symmetry



Slide credit: Kristen Grauman

Common Fate



Image credit: Arthus-Bertrand (via F. Durand)



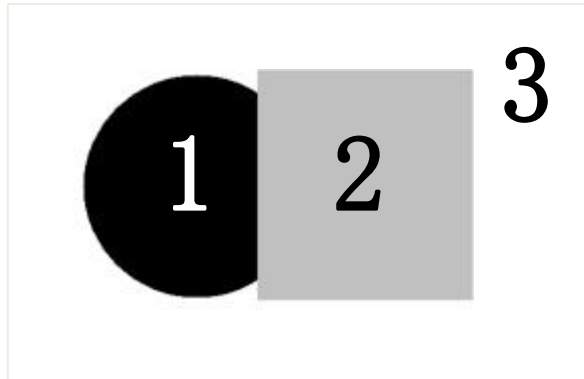
Slide credit: Kristen Grauman

Proximity

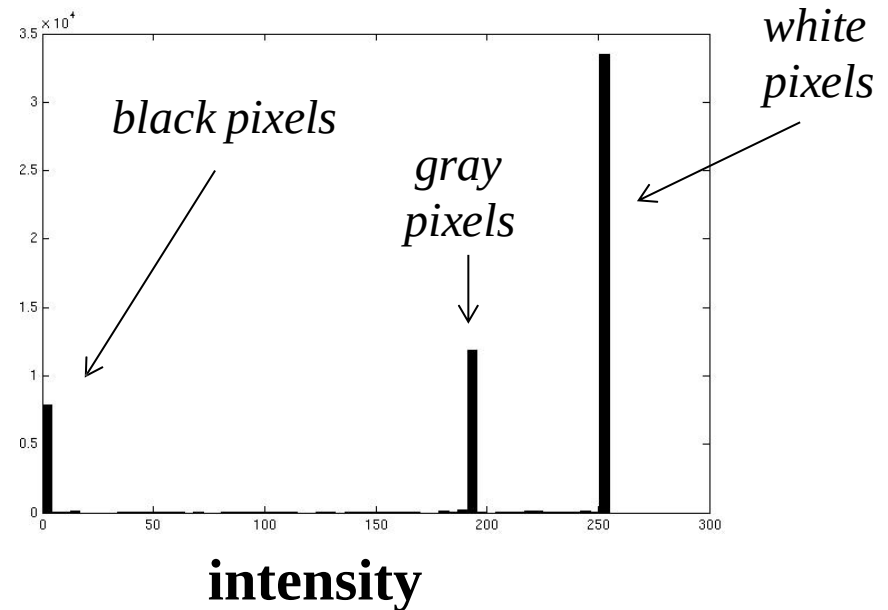


Slide credit: Kristen Grauman

Image Segmentation: Toy Example

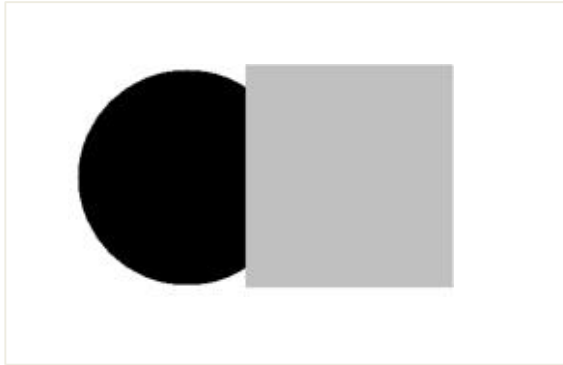


input image

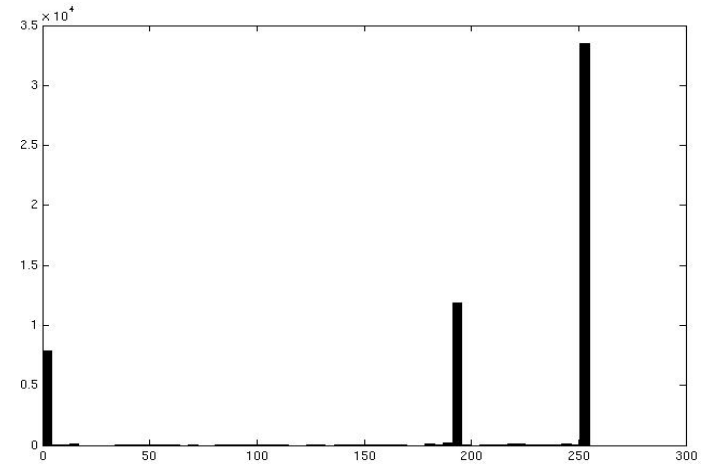


- These intensities define the three groups.
- We could label every pixel in the image according to which of these primary intensities it is.
- What if the image isn't quite so simple?

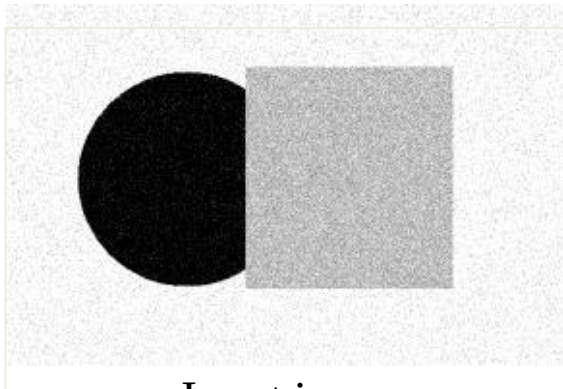
Image Segmentation: Toy Example



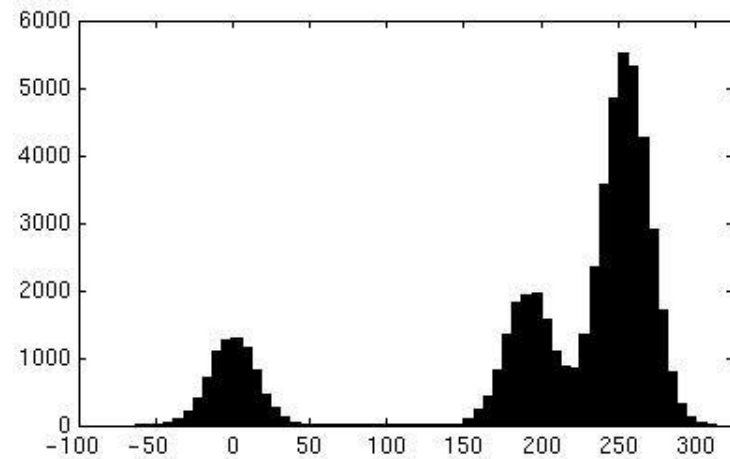
Input image



Intensity

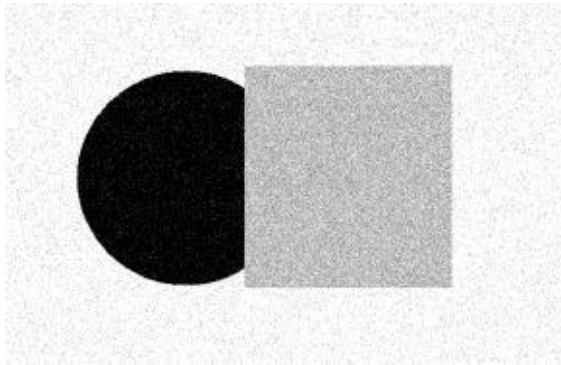


Input image

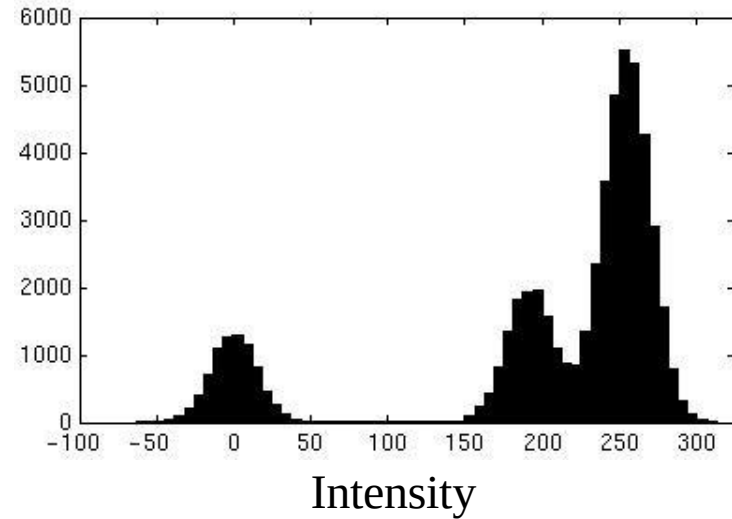


Intensity

Image Segmentation: Toy Example



Input image



- Now how to determine the three main intensities that define our groups?
- We need to cluster.



What we will learn today?

- Introduction to segmentation and clustering
- Agglomerative clustering
- K-means clustering
- Mean-shift clustering
- Expectation-Maximization (EM)

Clustering: distance measure

Clustering is an unsupervised learning method. Given items , $x_1, \dots, x_n \in \mathbb{R}^D$,the goal is to group them into clusters. We need a pairwise distance/similarity function between items, and sometimes the desired number of clusters.

When data (e.g. images, objects, documents) are represented by feature vectors, commonly used measures are:

- *Euclidean distance.*
- *cosine similarity.*

Defining Distance Measures

Let x and x' be two objects from the universe of possible objects.

The distance (similarity) is a real number denoted by $\text{sim}(x, x')$.

- The Euclidean distance is defined as

$$\text{dist}(x, x') = \sqrt{\sum (x_i - x'_i)^2}$$

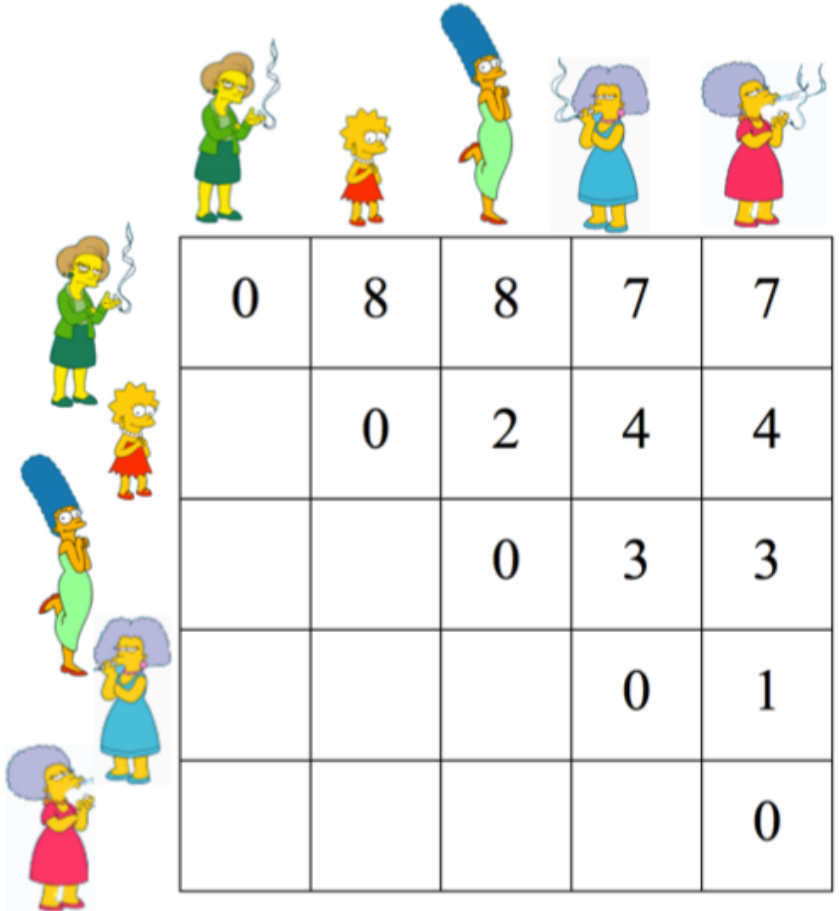
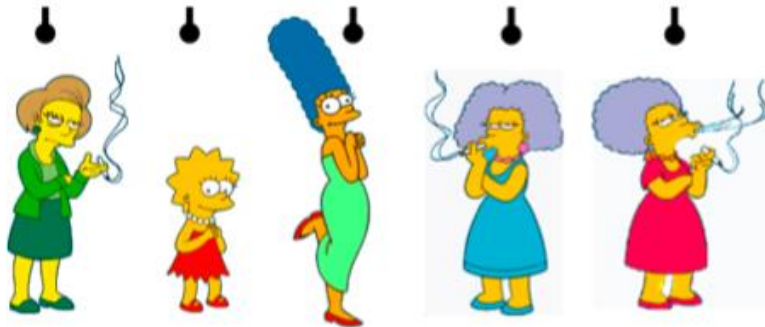
- In contrast, the cosine similarity measure would be

$$\begin{aligned} \text{sim}(x, x') &= \cos(\theta) \\ &= \frac{x^\top x'}{\|x\| \cdot \|x'\|} \\ &= \frac{x^\top x'}{\sqrt{x^\top x} \sqrt{x'^\top x'}}. \end{aligned}$$

Desirable Properties of a Clustering Algorithms

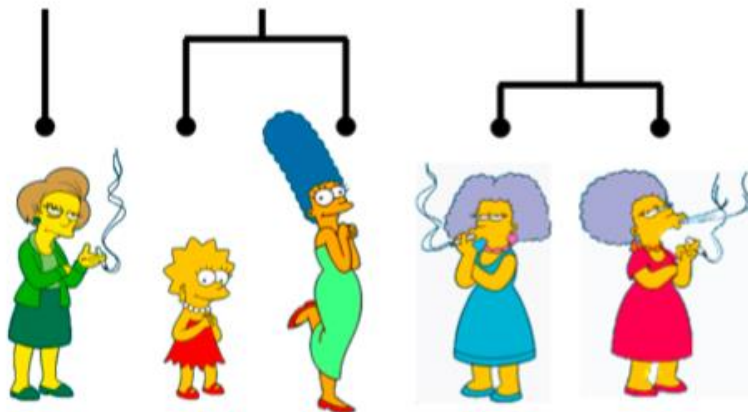
- Scalability (in terms of both time and space)
- Ability to deal with different data types
- Minimal requirements for domain knowledge to determine input parameters
- Interpretability and usability Optional
 - Incorporation of user-specified constraints

Animated example



[source](#)

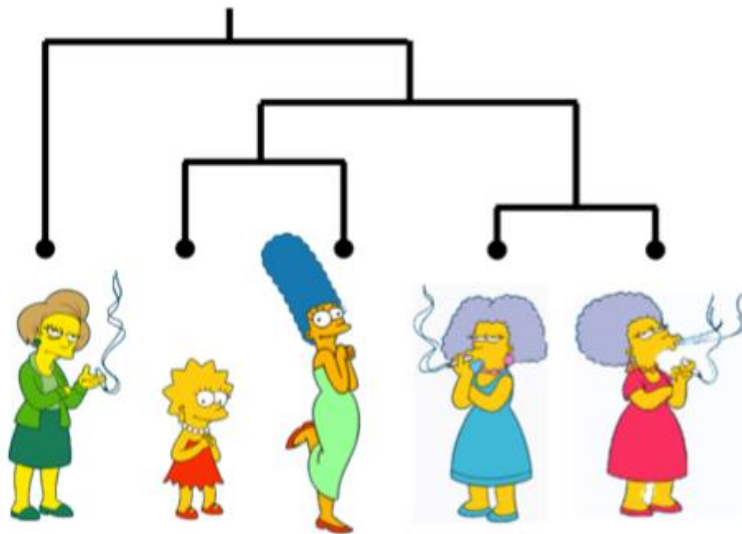
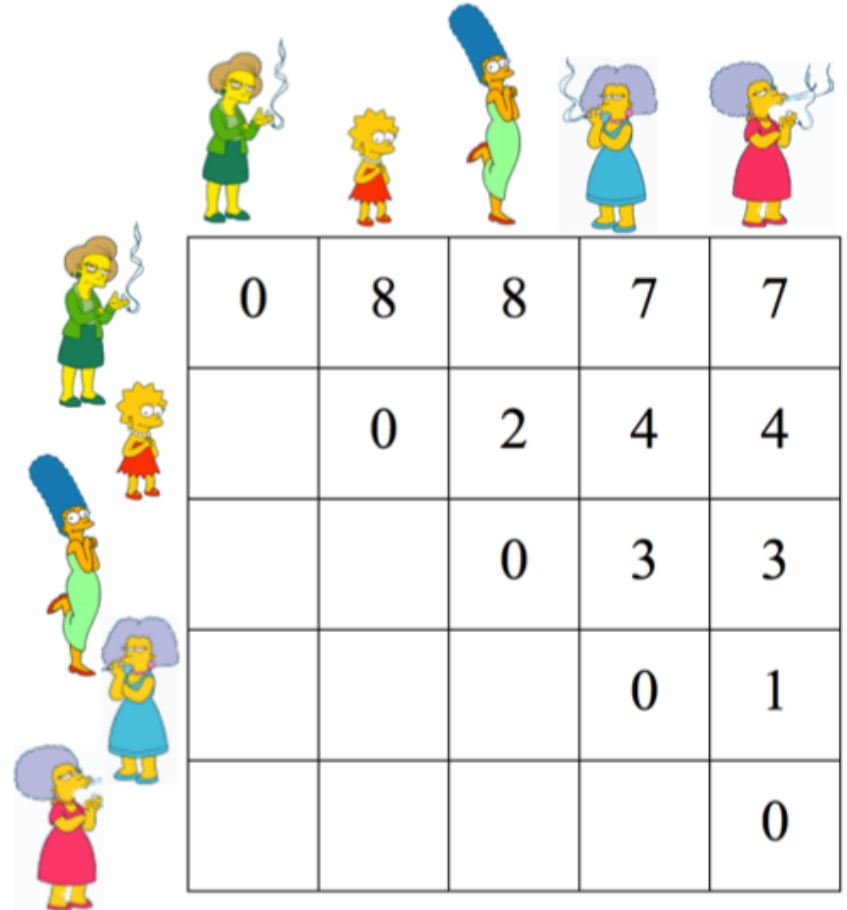
Animated example




				
0	8	8	7	7
				
	0	2	4	4
				
		0	3	3
				
			0	1
				0

[source](#)

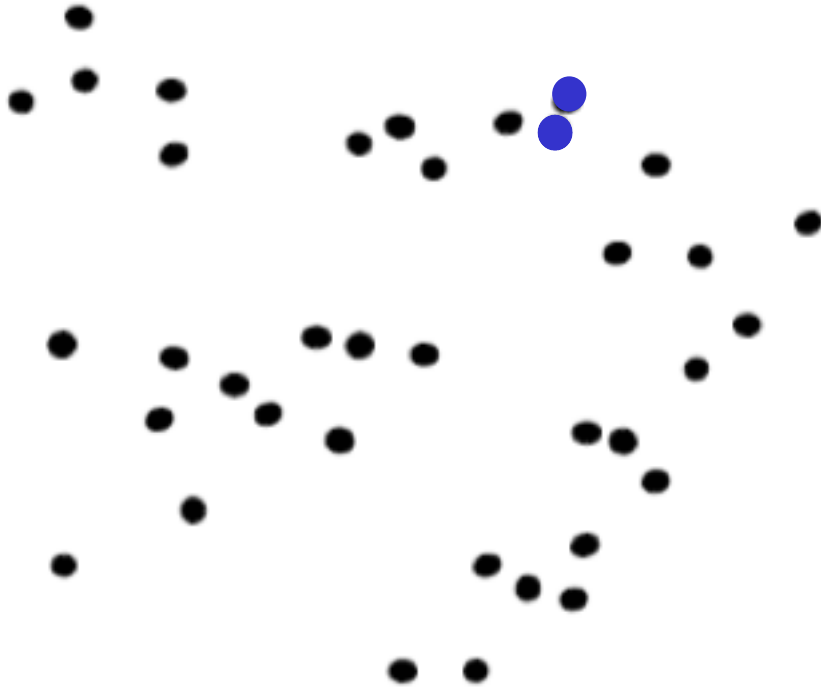
Animated example

				
0	8	8	7	7
				
	0	2	4	4
				
		0	3	3
				
			0	1
				
				0

[source](#)

Agglomerative clustering

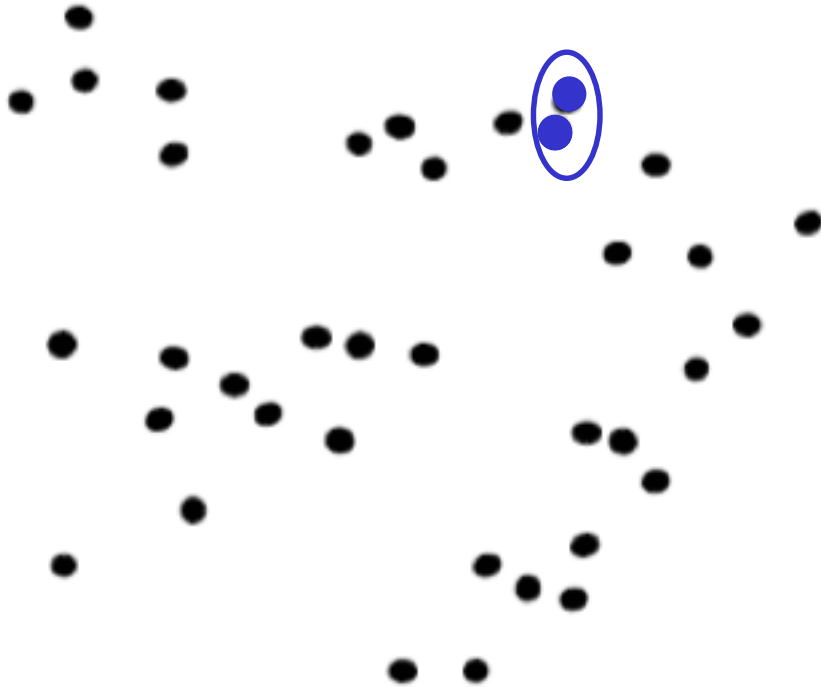


1. Say "Every point is its own cluster"

2. Find "most similar" pair of clusters



Agglomerative clustering



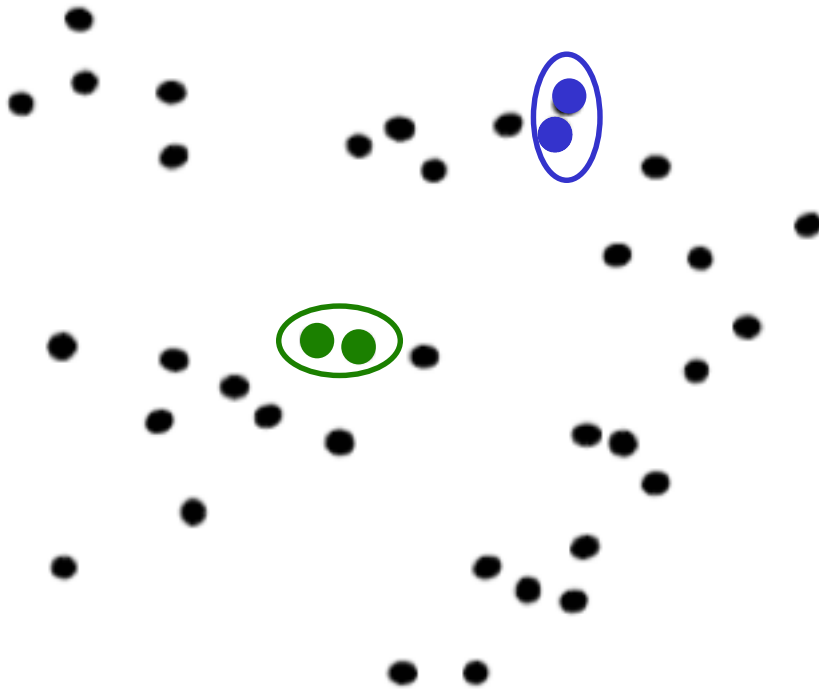
1. Say "Every point is its own cluster"

2. Find "most similar" pair of clusters

3. Merge it into a parent



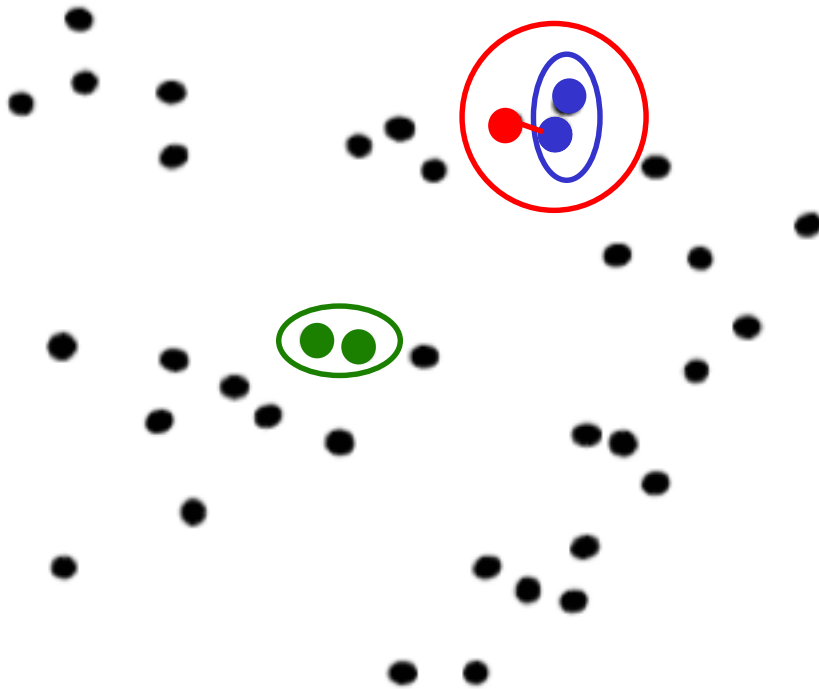
Agglomerative clustering



1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent
4. Repeat



Agglomerative clustering

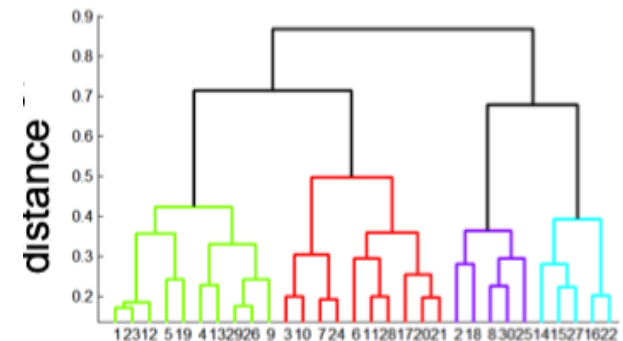
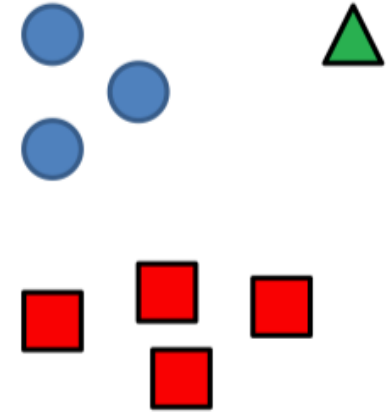


1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent
4. Repeat



Agglomerative clustering

- How to define cluster similarity?
 - Average distance between points,
 - maximum distance
 - minimum distance
 - Distance between means or medoids
- How many clusters?
 - Clustering creates a dendrogram (a tree)
 - Threshold based on max number of clusters or based on distance between merges



Agglomerative Hierarchical Clustering - Algorithm

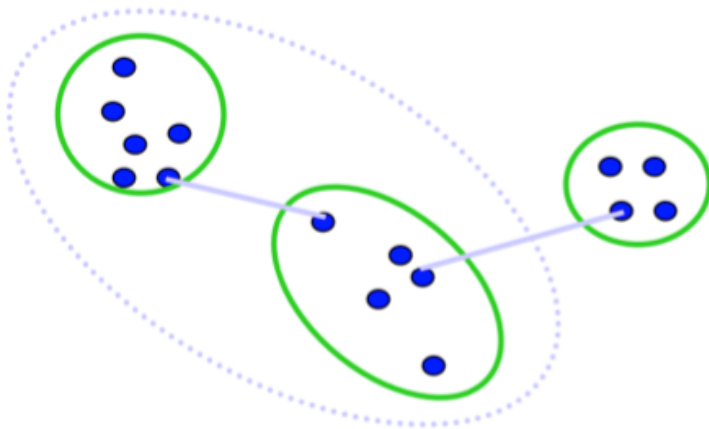
1. Initially each item $x_1, \dots, x_n$ is in its own cluster $C_1, \dots, C_n$.
2. Merge the nearest clusters, say C_i and C_j
3. Repeat until there is only one cluster .

Different measures of nearest clusters

Nearest Neighbor Clustering

Single Link

- $d(C_i, C_j) = \min_{x \in C_i, x' \in C_j} d(x, x')$. This is known as *single-linkage*. It is equivalent to the minimum spanning tree algorithm. One can set a threshold and stop clustering once the distance between clusters is above the threshold. Single-linkage tends to produce long and skinny clusters.



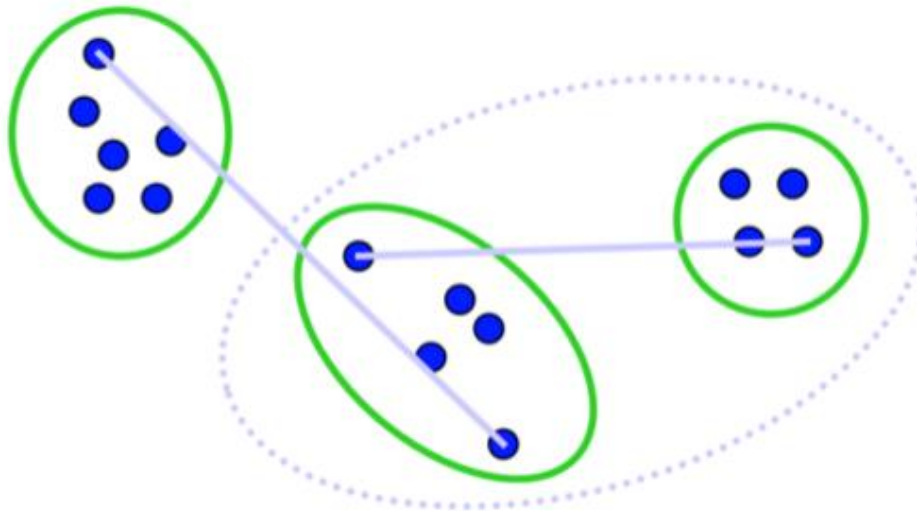
Long, skinny clusters

Different measures of nearest clusters

Furthest neighbor clustering

Complete Link

- $d(C_i, C_j) = \max_{x \in C_i, x' \in C_j} d(x, x')$. This is known as *complete-linkage*. Clusters tend to be compact and roughly equal in diameter.

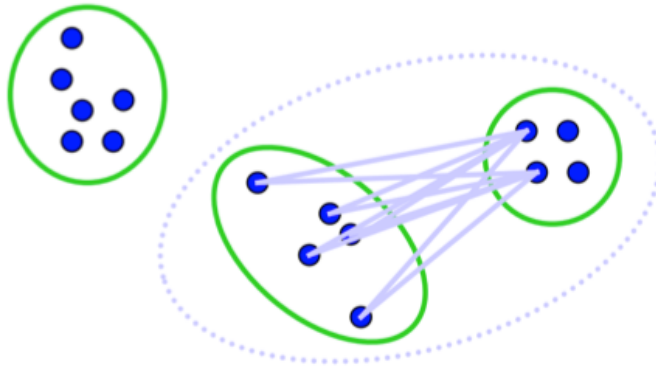


Tight clusters

Different measures of nearest clusters

Average Link

- $d(C_i, C_j) = \frac{\sum_{x \in C_i, x' \in C_j} d(x, x')}{|C_i| \cdot |C_j|}$. This is the average distance between items. Somewhere between single-linkage and complete-linkage.



Robust against noise.

Conclusions: Agglomerative Clustering

- Good
 - Simple to implement, widespread application.
 - Clusters have adaptive shapes.
 - Provides a hierarchy of clusters.
 - No need to specify number of clusters in advance.

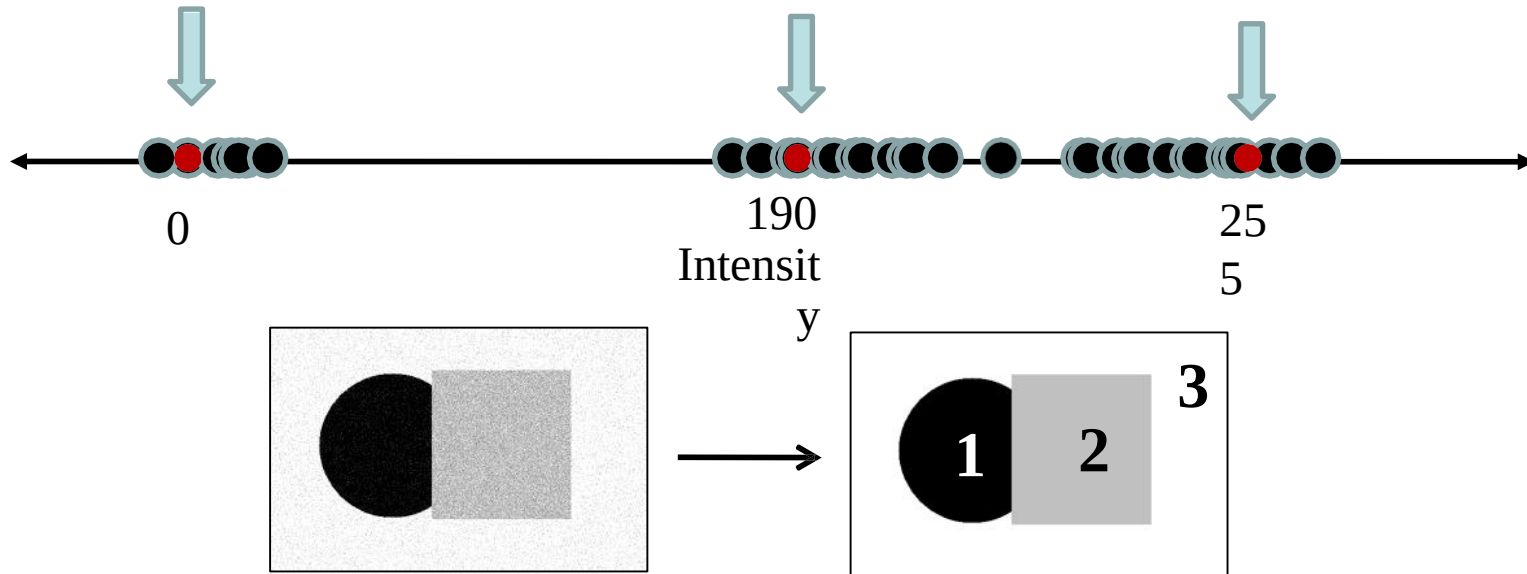
- Bad
 - May have imbalanced clusters.
 - Still have to choose number of clusters or threshold.
 - Does not scale well. Runtime of $O(n^3)$.
 - Can get stuck at a local optima.



What we will learn today?

- Introduction to segmentation and clustering
- Agglomerative clustering
- **K-means clustering**
- Mean-shift clustering
- Expectation-Maximization (EM)

K-means clustering: introduce



- Goal: choose three “centers” as the representative intensities, and label every pixel according to which of these centers it is nearest to.
- Best cluster centers are those that minimize Sum of Square Distance (SSD) between all points and their nearest cluster center c_i

$$SSD = \sum_{\text{Cluster } i} \sum_{x \in \text{cluster } i} (x - c_i)^2$$

K-means clustering: summary

Goal: cluster to minimize variance in data given cluster

$$c^*, \delta^* = \underset{c, \delta}{\operatorname{argmin}} \frac{1}{N} \sum_j^N \sum_i^K \delta_{ij} (c_i - x_j)^2$$

Cluster center

Data

Whether x_j is assigned to c_i

K-means clustering: algorithm

1. Initialize ($t = 0$): cluster centers $c_1, \dots, c_K$

2. Compute δ^t : assign each point to the closest center

– δ^t denotes the set of assignment for each x_j to cluster C_i at iteration t

$$\delta^t = \underset{\delta}{\operatorname{argmin}} \frac{1}{N} \sum_j \sum_i^K \delta_{ij}^{t-1} (c_i^{t-1} - x_j)^2$$

3. Computer c^t : update cluster centers as the mean of the points

$$c^t = \underset{c}{\operatorname{argmin}} \frac{1}{N} \sum_j \sum_i^K \delta_{ij}^t (c_i^{t-1} - x_j)^2$$

4. Update $t = t + 1$, Repeat Step 2-3 till stopped



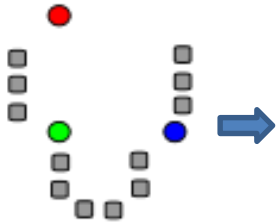
K-means clustering: detail

1. Initialize ($t = 0$): cluster centers $c_1, \dots, c_K$
 - Commonly used: random initialization
2. Compute δ^t : assign each point to the closest center
 - Typical distance measure:
 - Euclidean $\text{sim}(x, x') = \|x - x'\|^2$
 - Cosine $\text{sim}(x, x') = x^T x' / (\|x\| \cdot \|x'\|)$
 - Others
3. Computer c^t : update cluster centers as the mean of the points

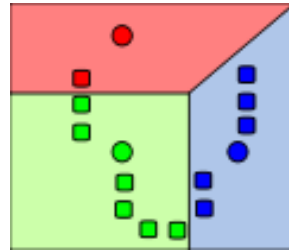
$$c^t = \underset{c}{\operatorname{argmin}} \frac{1}{N} \sum_j^N \sum_i^K \delta_{ij}^t \left(c_i^{t-1} x_j \right)^2$$

4. Update $t = t + 1$, Repeat Step 2-3 till stopped
 - c^t doesn't change anymore.

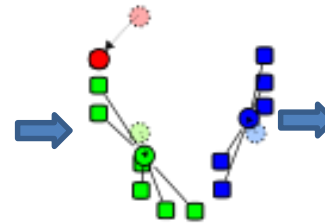
K-means clustering: example



1. Initialize
Cluster Centers



2. Assign Points to
Clusters



3. Re-compute
Means



Repeat (2) and (3)

- Java demo:

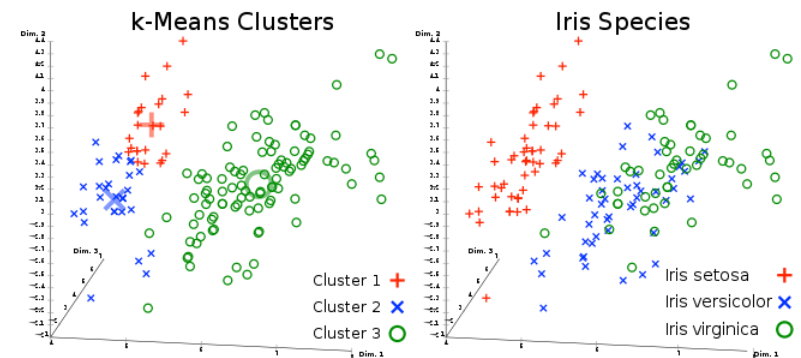
http://home.dei.polimi.it/matteucc/Clustering/tutorial_html/AppletKM.html

K-means clustering: analysis

- Converges to a *local minimum* solution
 - Initialize multiple runs



- Better fit for spherical data



- Need to pick K (# of clusters)

K-means++ clustering

- Can we prevent arbitrarily bad local minima?

K-means++: Try to separate the initial clustering centers as much as possible

1. Randomly choose first center.
2. Calculate the shortest distance, $D(x)$ between each point and the chosen centers.
3. Calculate the prob. of being selected as the new center for x ,
$$P(x) = \frac{D(x)^2}{\sum_x D(x)^2} .$$
4. Select the new center according to these prob.
5. Repeat step2-4 until K centers are selected

Feature space

- Grouping pixels based on **intensity** similarity.
- The feature space is 1D



Original image



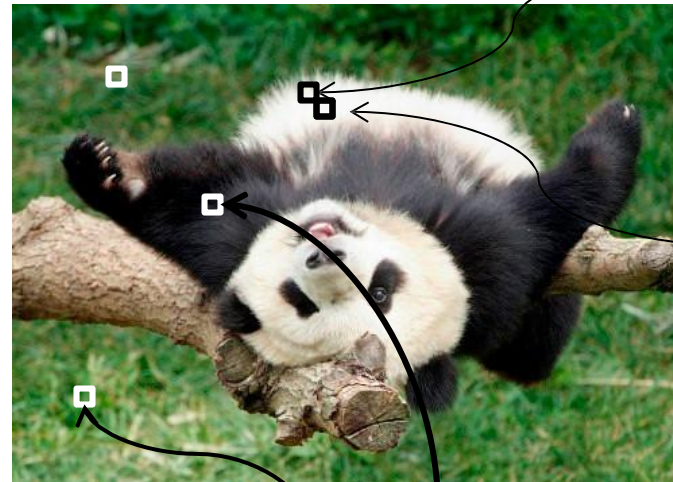
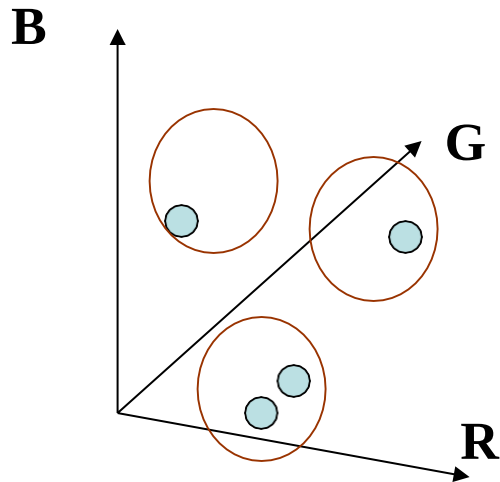
2 clusters



3 clusters

Feature space

- Grouping pixels based on **color** similarity
- The feature space is 3D



$\begin{pmatrix} R=255 \\ G=200 \\ B=250 \end{pmatrix}$

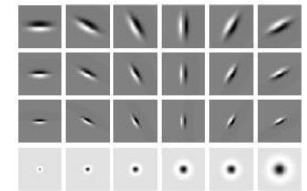
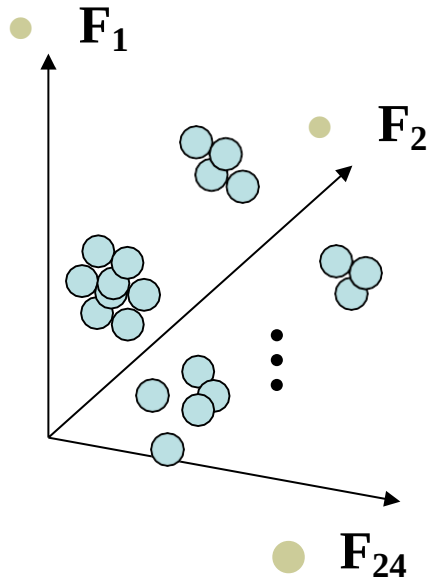
$\begin{pmatrix} R=245 \\ G=220 \\ B=248 \end{pmatrix}$

$\begin{pmatrix} R=15 \\ G=189 \\ B=2 \end{pmatrix}$

$\begin{pmatrix} R=3 \\ G=12 \\ B=2 \end{pmatrix}$

Feature space

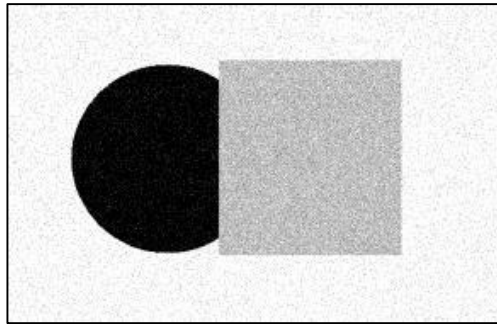
- Grouping pixels based on **texture** similarity
- The feature space is 24D



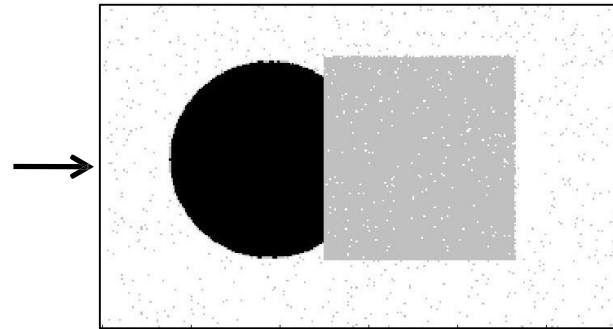
Filter bank of 24 filters

Feature space

- Assigning a cluster label per pixel may yield outliers.



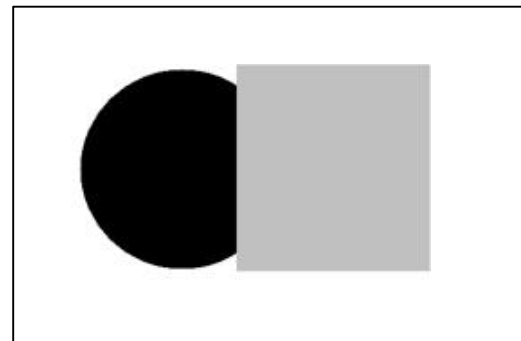
Original



Labeled by cluster center's intensity

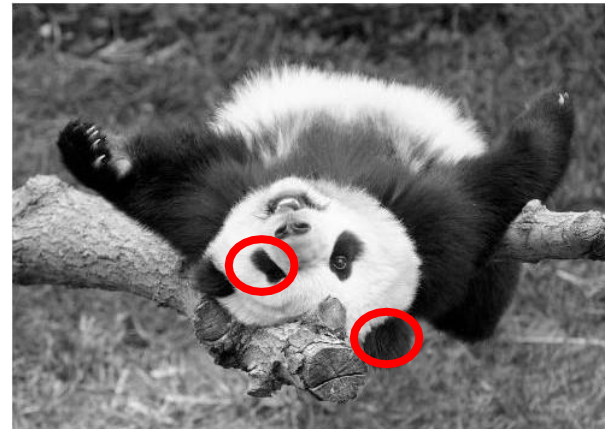
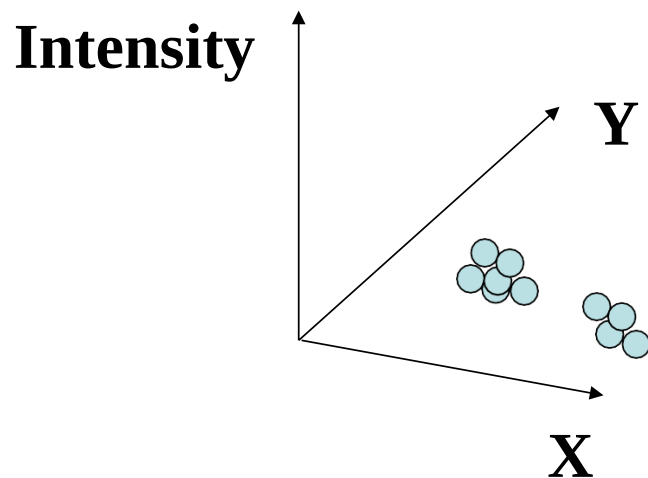


- How to ensure that the clustering results are spatially smooth?



Feature space

- Grouping pixels based on *intensity+position* similarity.
- The feature space is 3D



- Way to encode both similarity and proximity.

K-Means Clustering Results

- K-means clustering based on intensity or color is essentially vector quantization of the image attributes
 - Clusters don't have to be spatially coherent

Image

Intensity-based cluster

Color-based clusters

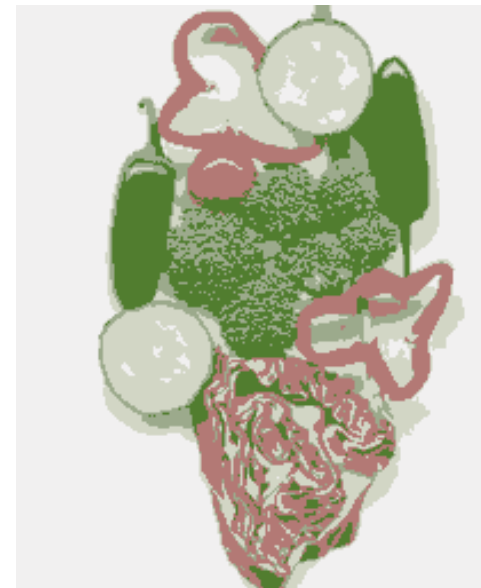


Image source: Forsyth & Ponce

K-Means Clustering Results

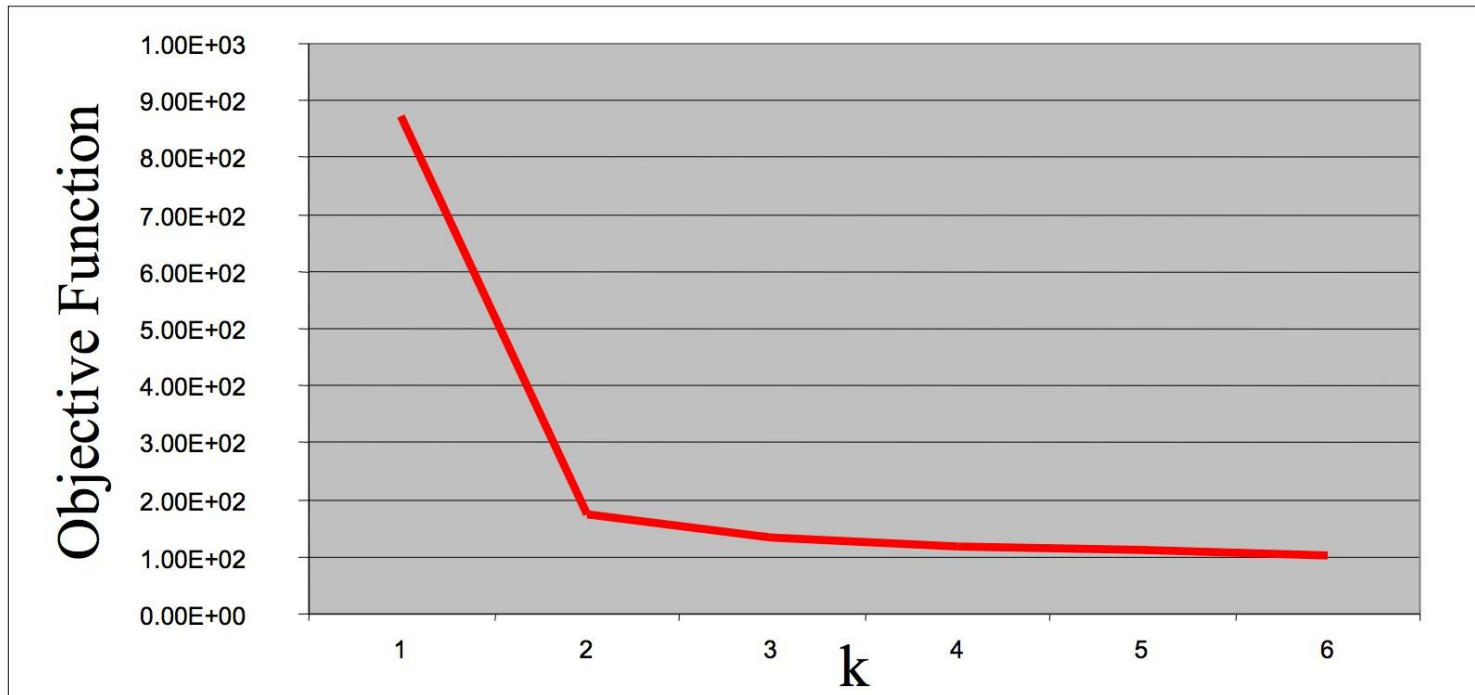
- Clustering based on (r, g, b, x, y) values enforces more spatial coherence



Image source: Forsyth & Ponce

How to evaluate clusters?

- **Determine the number of clusters**
 - Use “elbow finding” (or “knee finding”). Try different numbers of clusters in a validation set and look at performance.



How to evaluate clusters?

- **Evaluate clustering quality**

- Internal method:

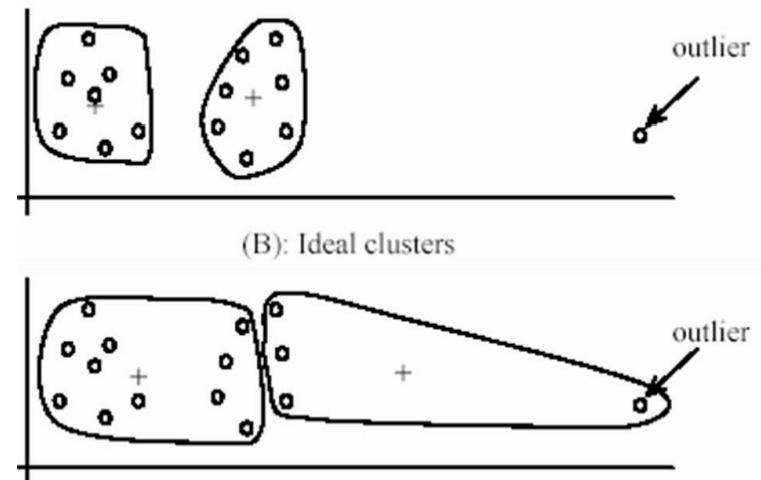
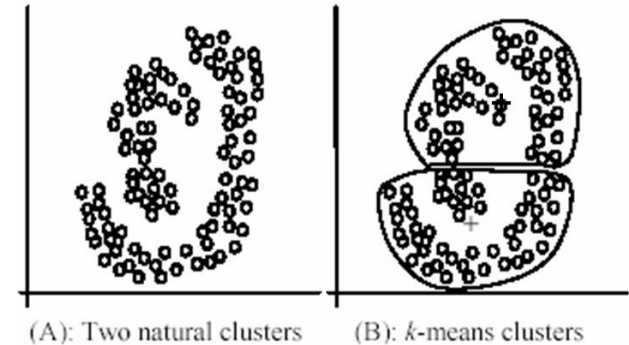
- unsupervised, evaluate the class cohesion and class dispersion
- silhouette coefficient; Davies-Bouldin Index(DBI); Dunn Index(DI)

- External method:

- supervised, evaluate the consistency between clustering results and benchmark data(labeled data)
- Bcubed precision, Bcubed recall
- Jaccard Coefficient(JC), Fowlkes and Mallows Index(FMI), Rand Index(RI)

K-Means pros and cons

- Pros
 - Finds cluster centers that minimize conditional variance (good representation of data)
 - Simple and fast, Easy to implement
- Cons
 - Need to choose K
 - Prone to local minima
 - Fit bad for non spherical data
 - Sensitive to outliers
- Usage
 - Unsupervised clustering
 - Rarely used for pixel segmentation



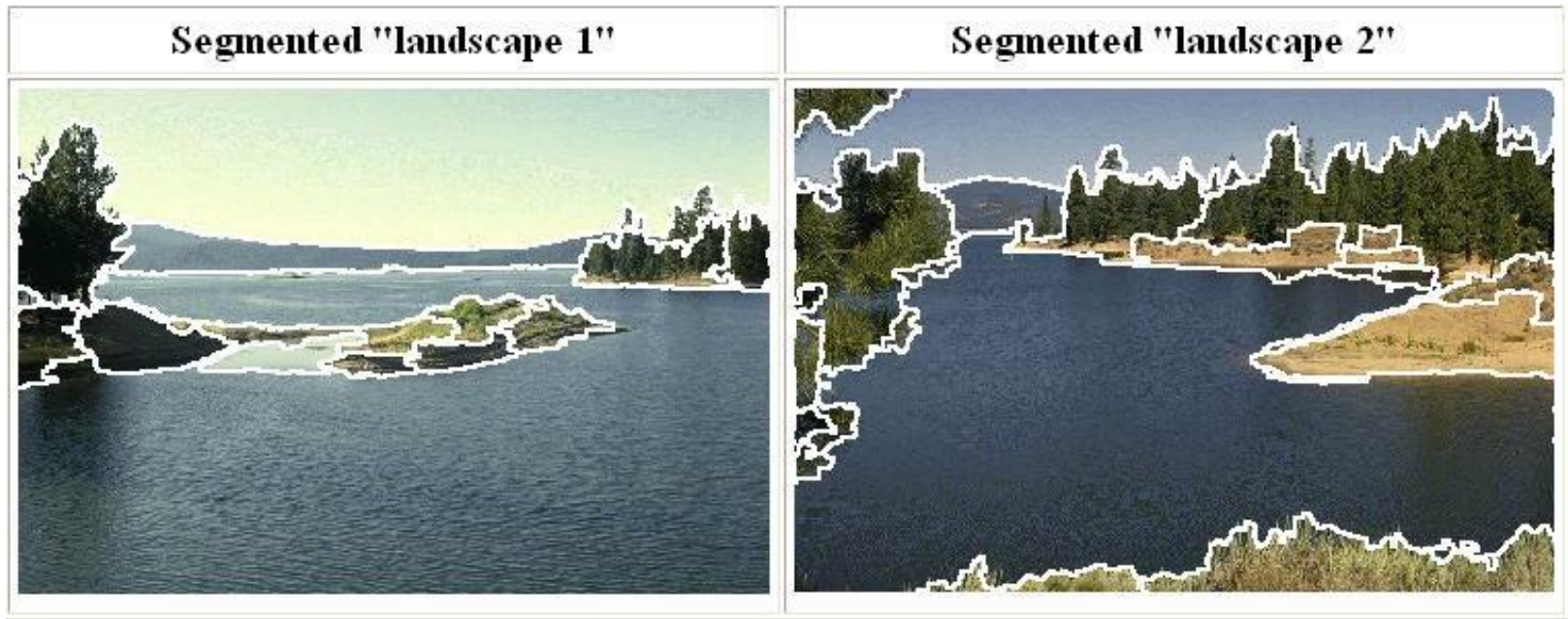


What we will learn today?

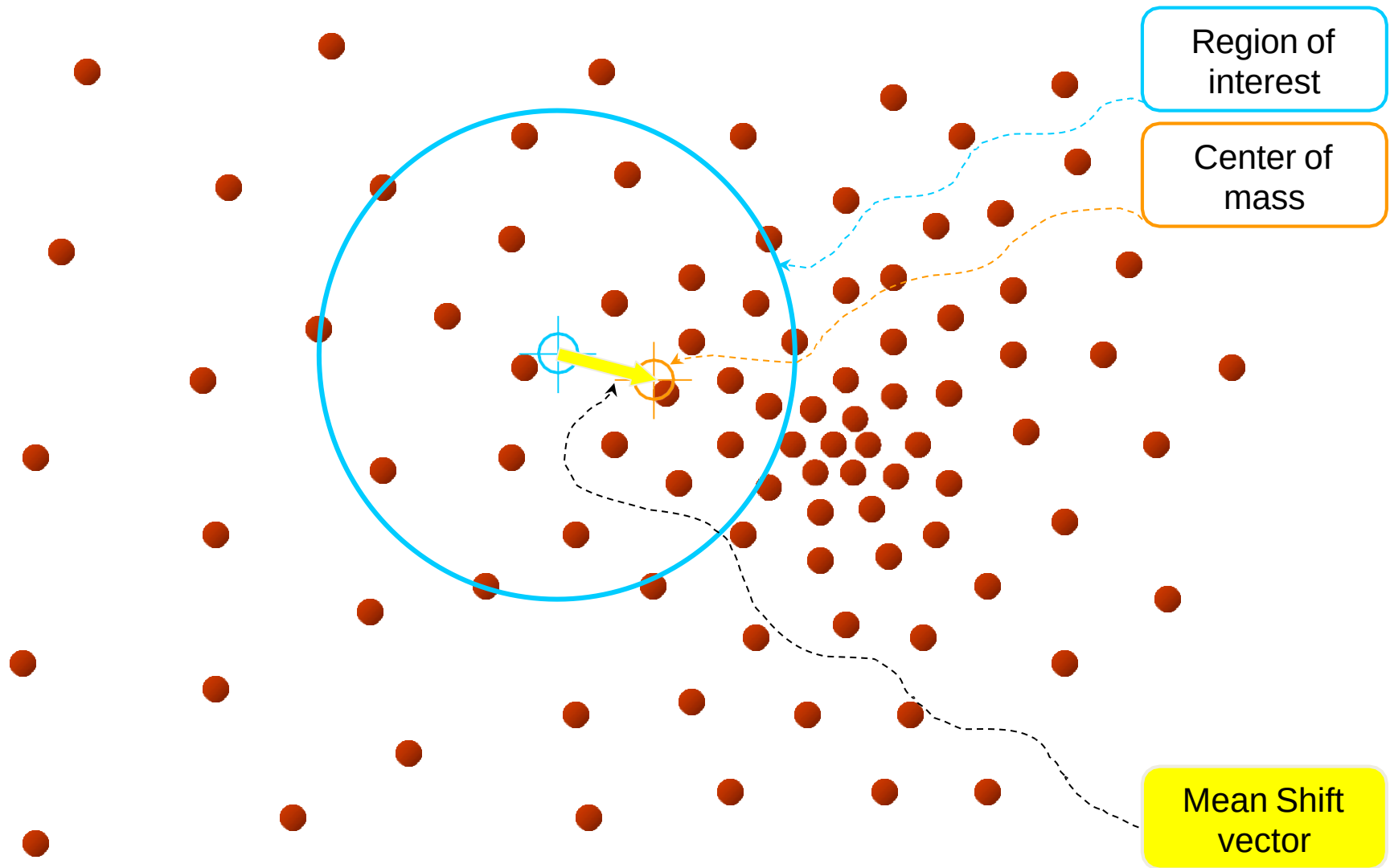
- Introduction to segmentation and clustering
- Agglomerative clustering
- K-means clustering
- Mean-shift clustering
- Expectation-Maximization (EM)

Mean-Shift Segmentation

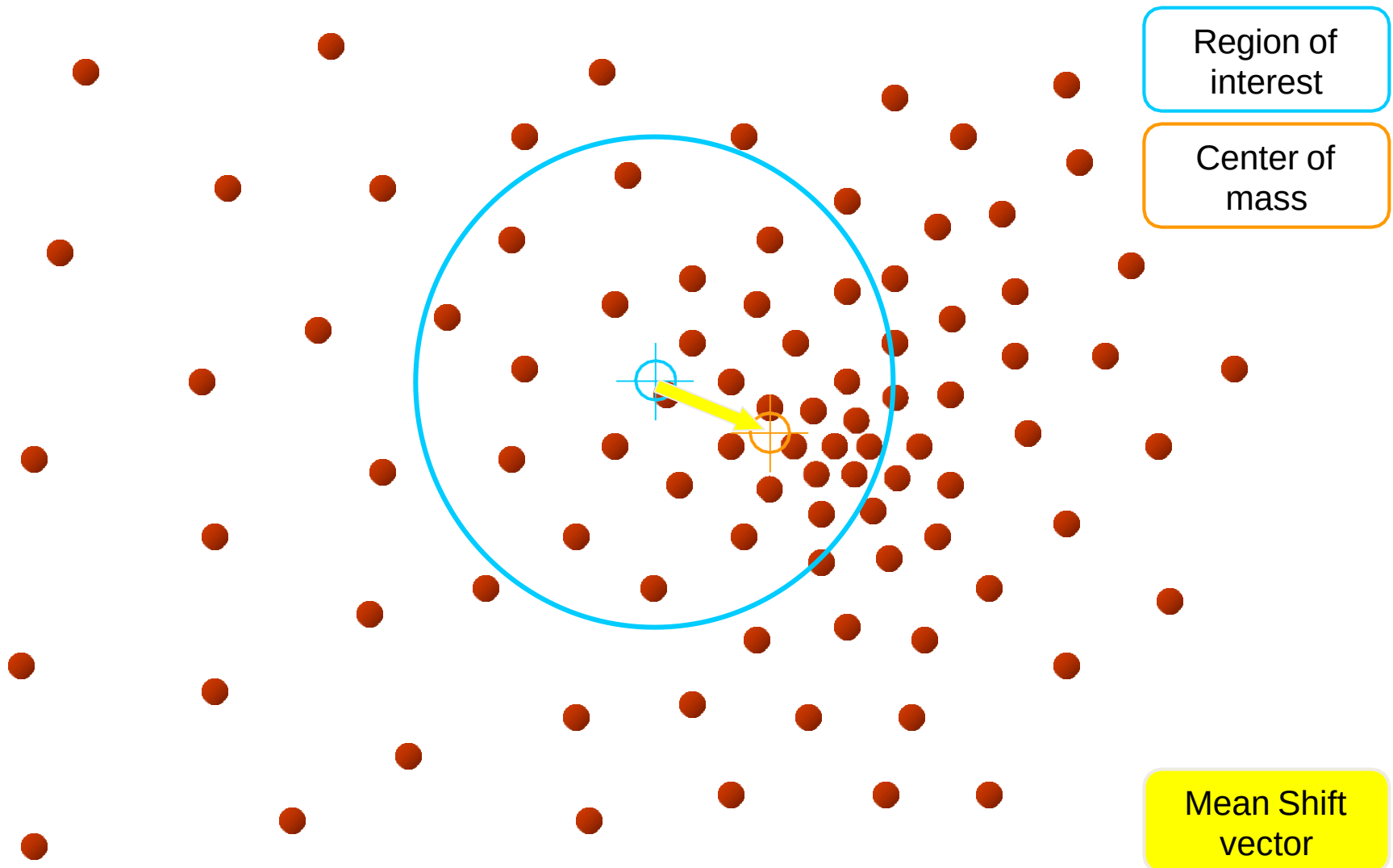
- An advanced and versatile technique for clustering-based segmentation



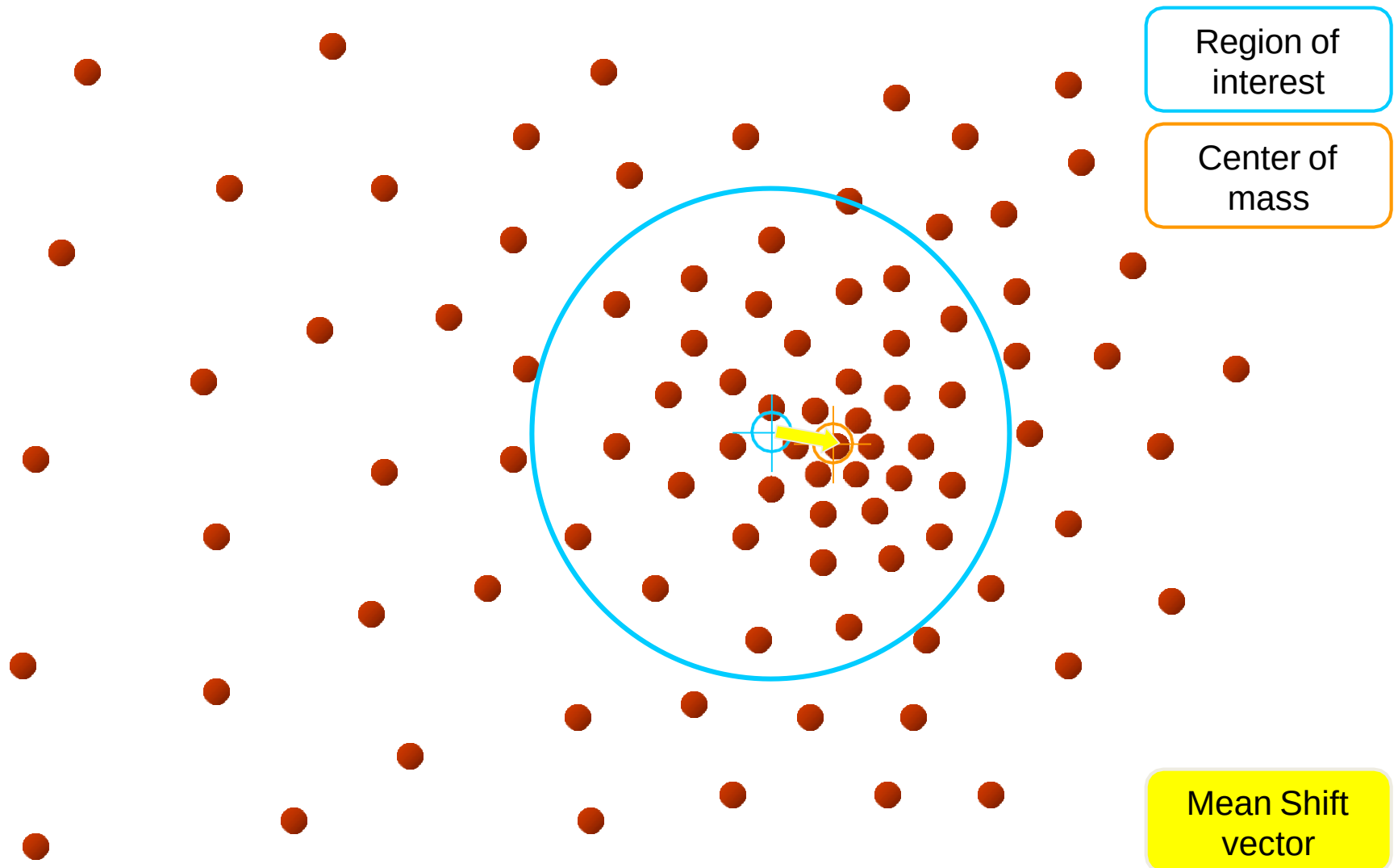
Mean-Shift



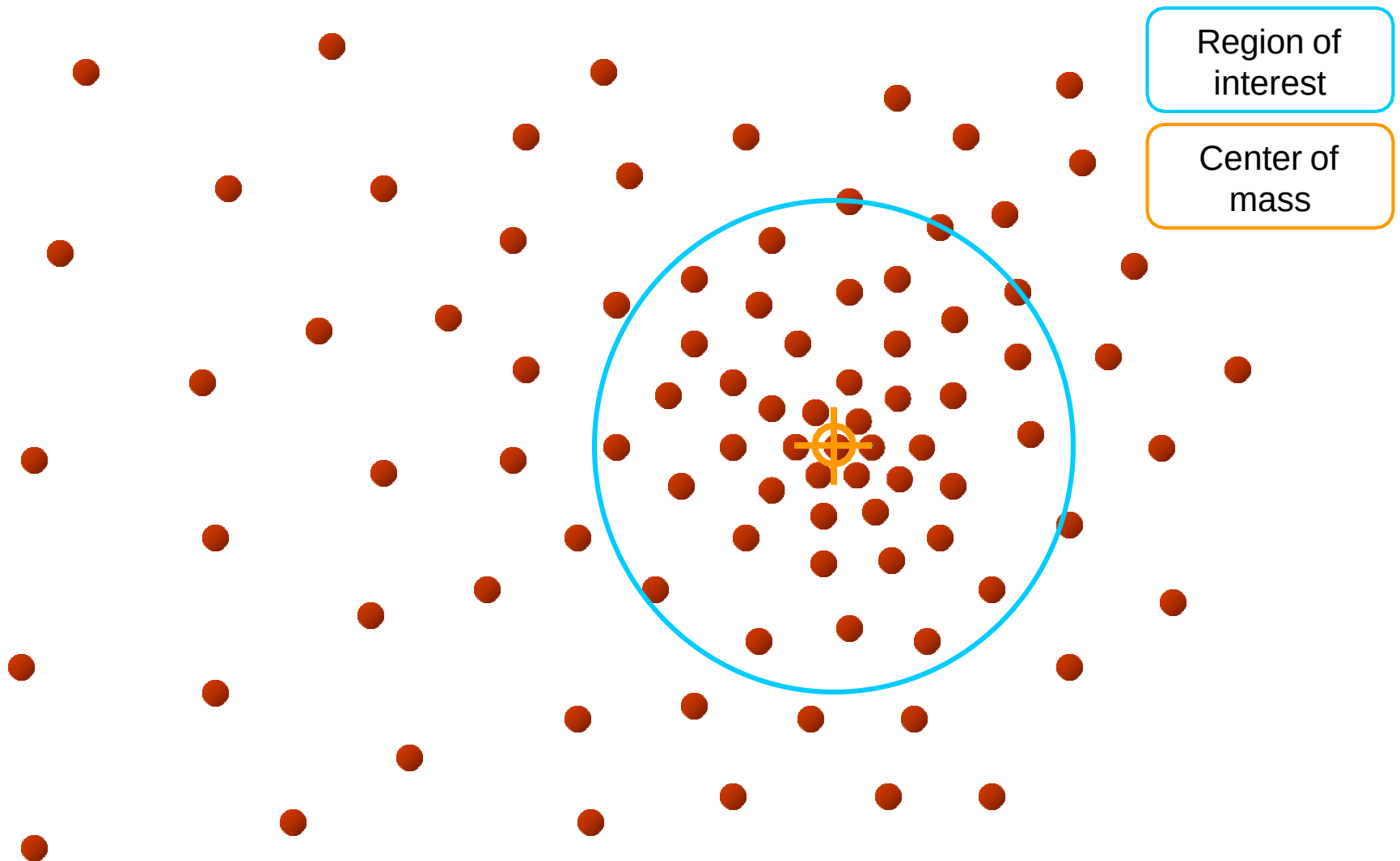
Mean-Shift



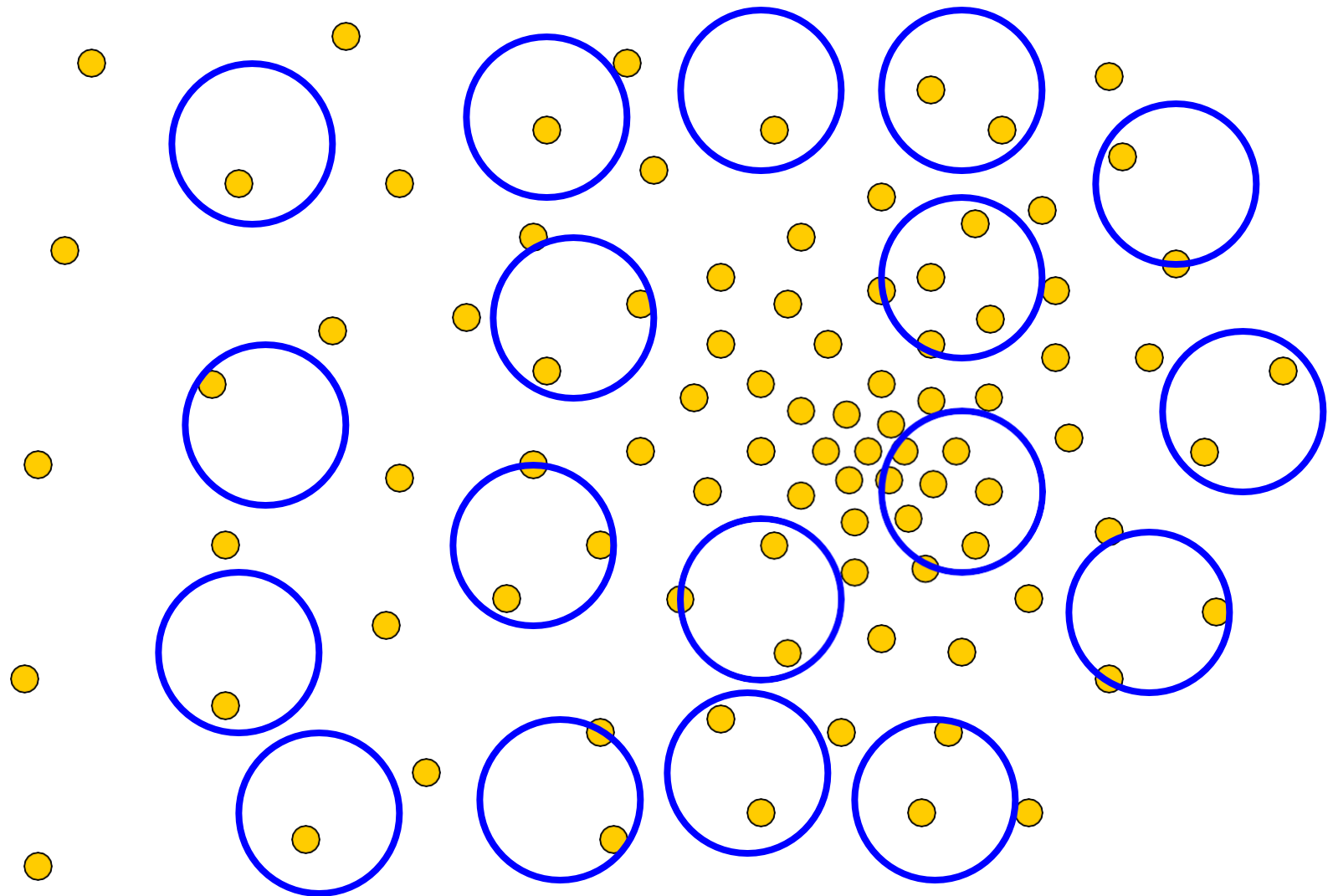
Mean-Shift



Mean-Shift



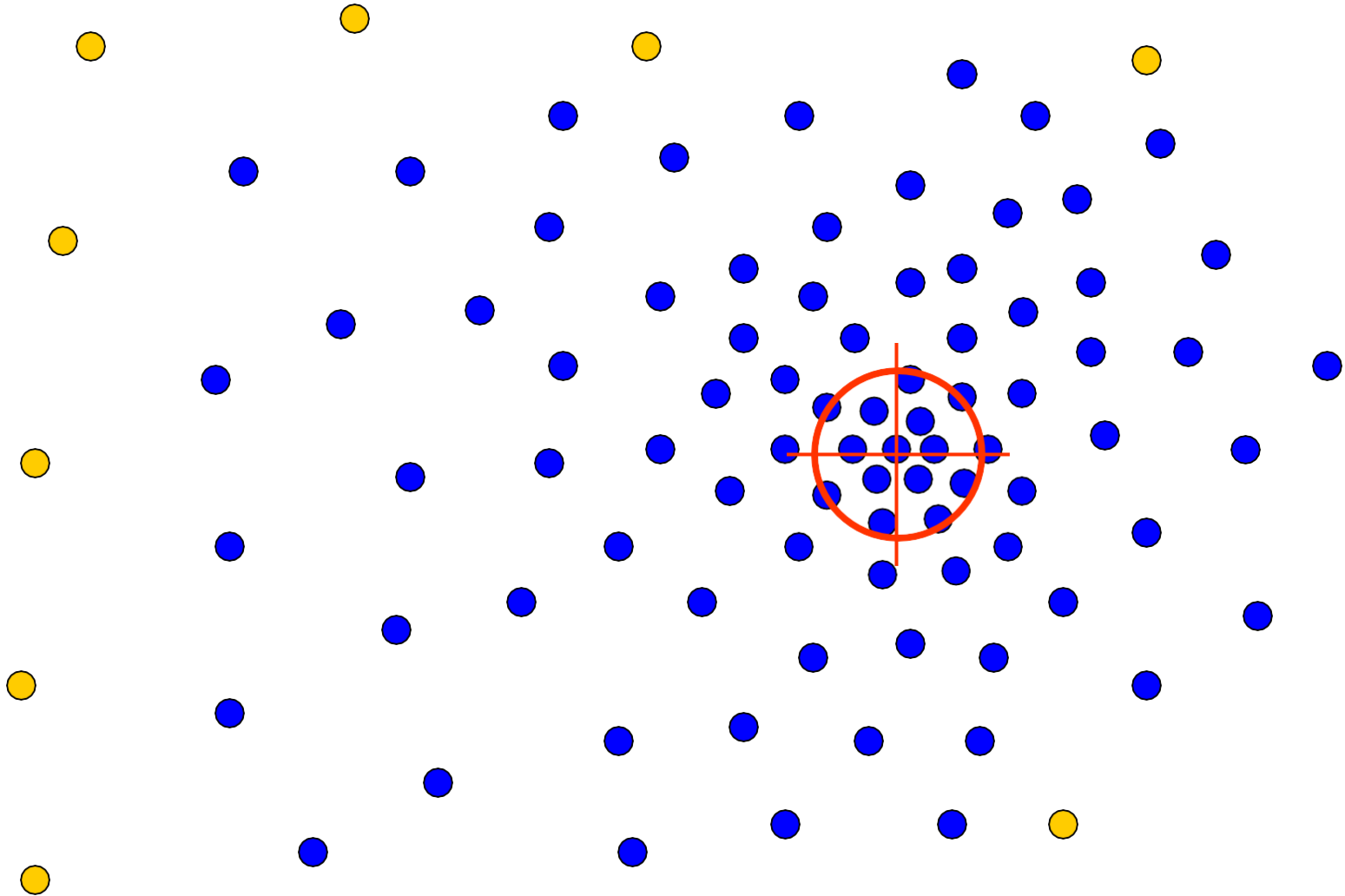
Real Modality Analysis



Tessellate the space with windows

Run the procedure in parallel

Real Modality Analysis

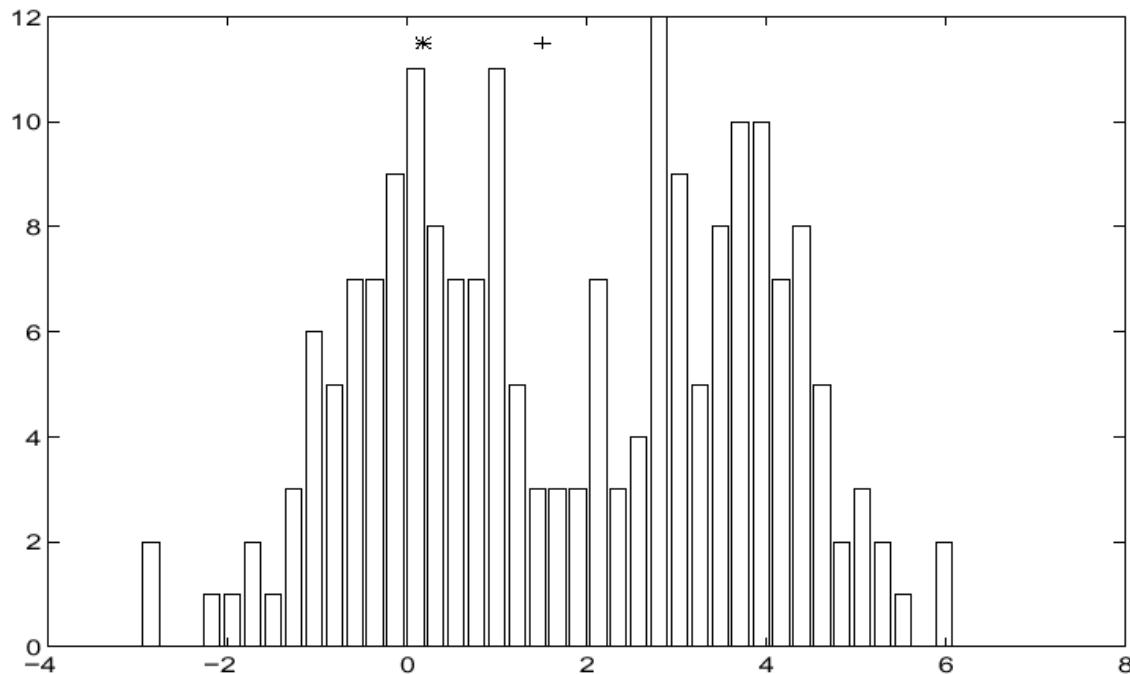


The **blue** data points were traversed by the windows towards the mode.

Mean-Shift Algorithm

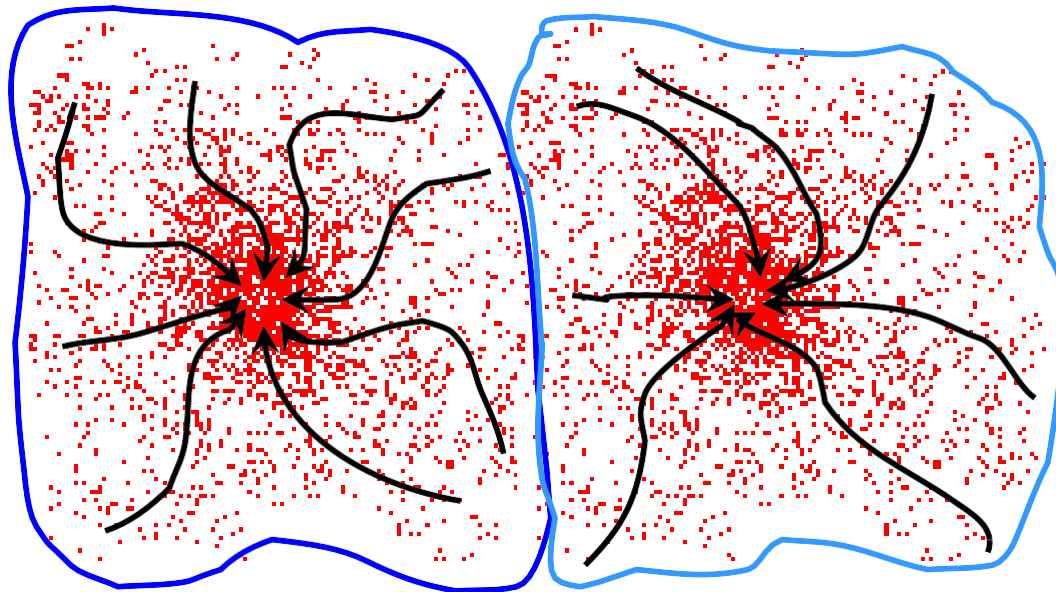
- Iterative Mode Search

1. Initialize random seed, and window W
2. Calculate center of gravity (the “mean”) of W : $\sum_{x \in W} x H(x)$
3. Shift the search window to the mean
4. Repeat Step 2 and Step 3 until convergence



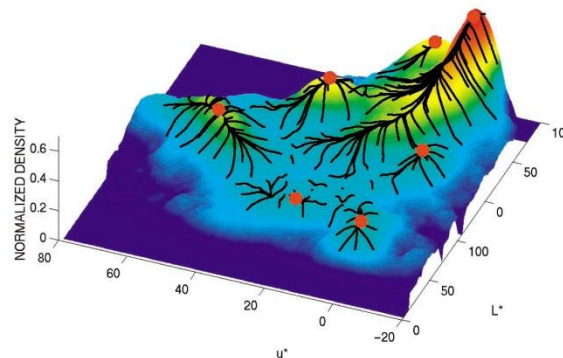
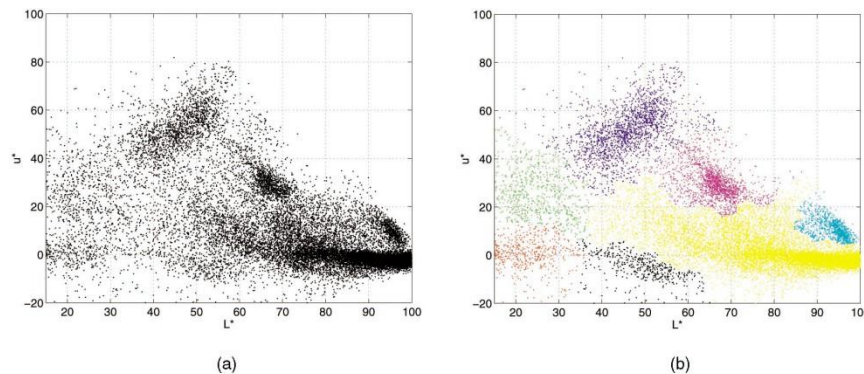
Mean-Shift Clustering

- Cluster: all data points in the attraction basin of a mode
- Attraction basin: the region for which all trajectories lead to the same mode

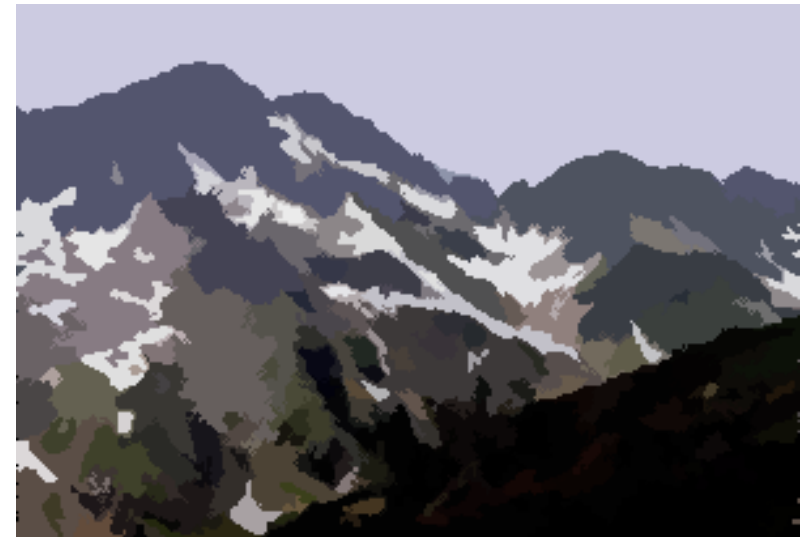


Mean-Shift Clustering/Segmentation

- Find features (color, gradients, texture, etc)
- Initialize windows at individual pixel locations
- Perform mean shift for each window until convergence
- Merge windows that end up near the same “peak” or mode



Mean-Shift Segmentation Results



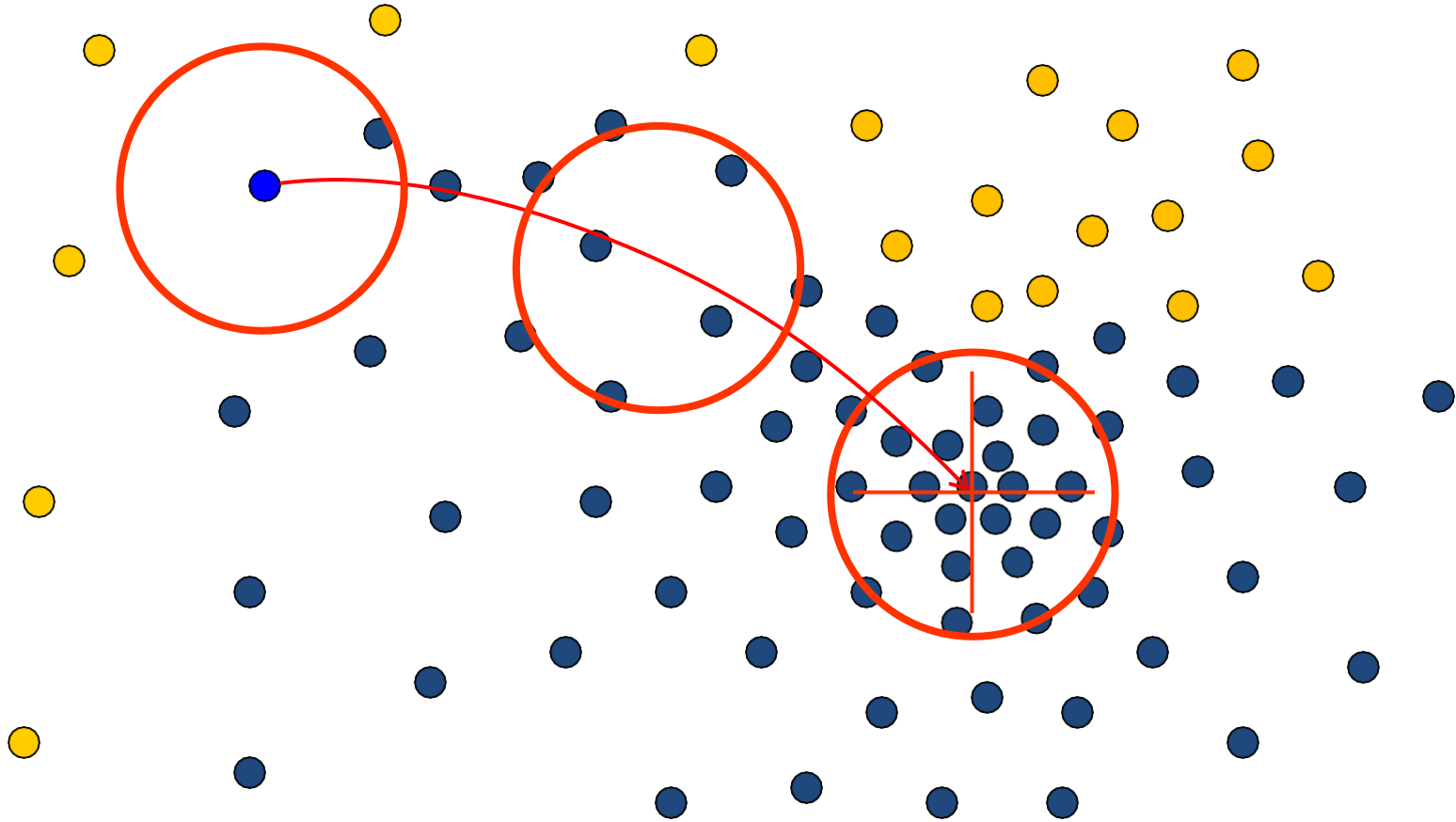
More Results



More Results

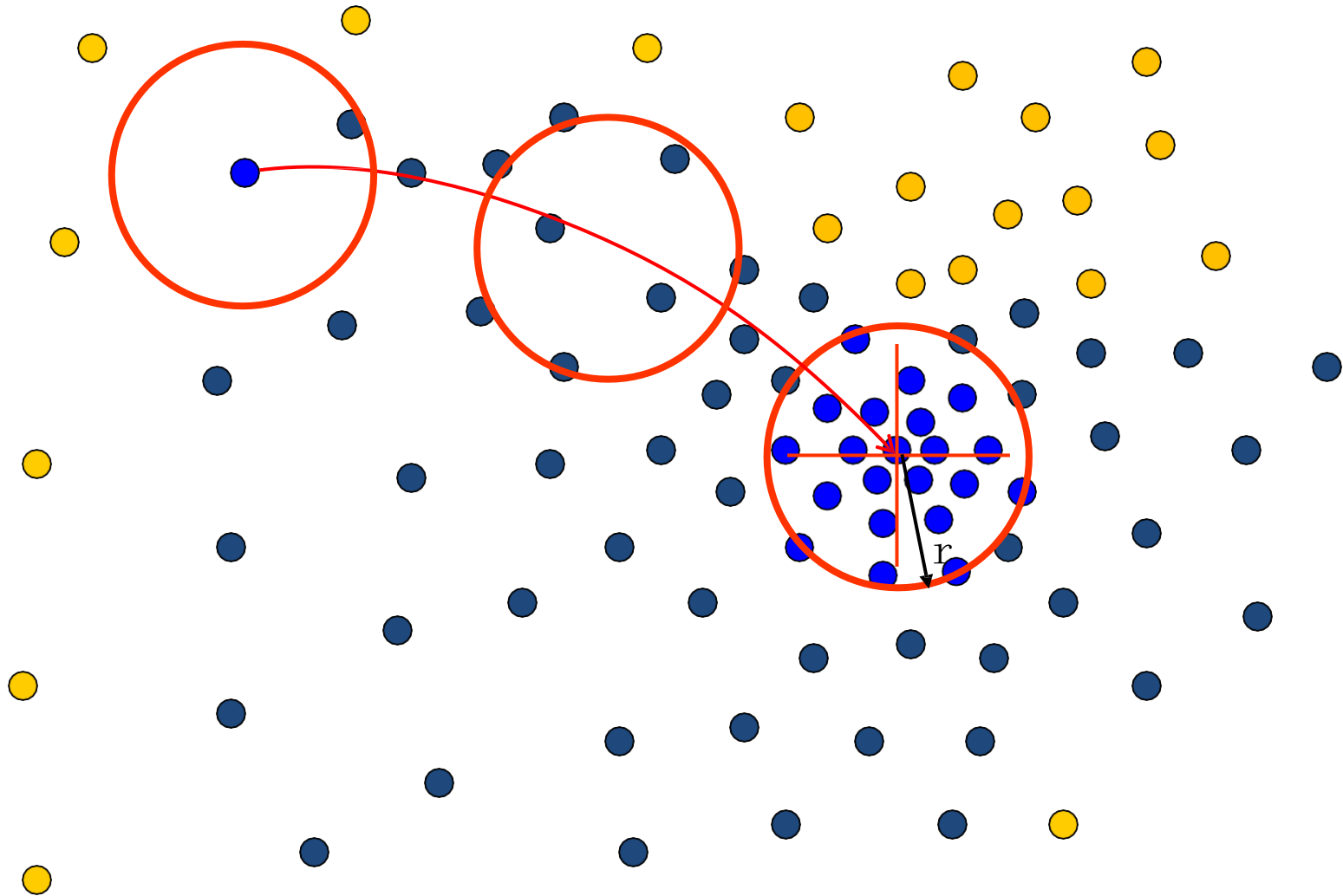


Problem: Computational Complexity



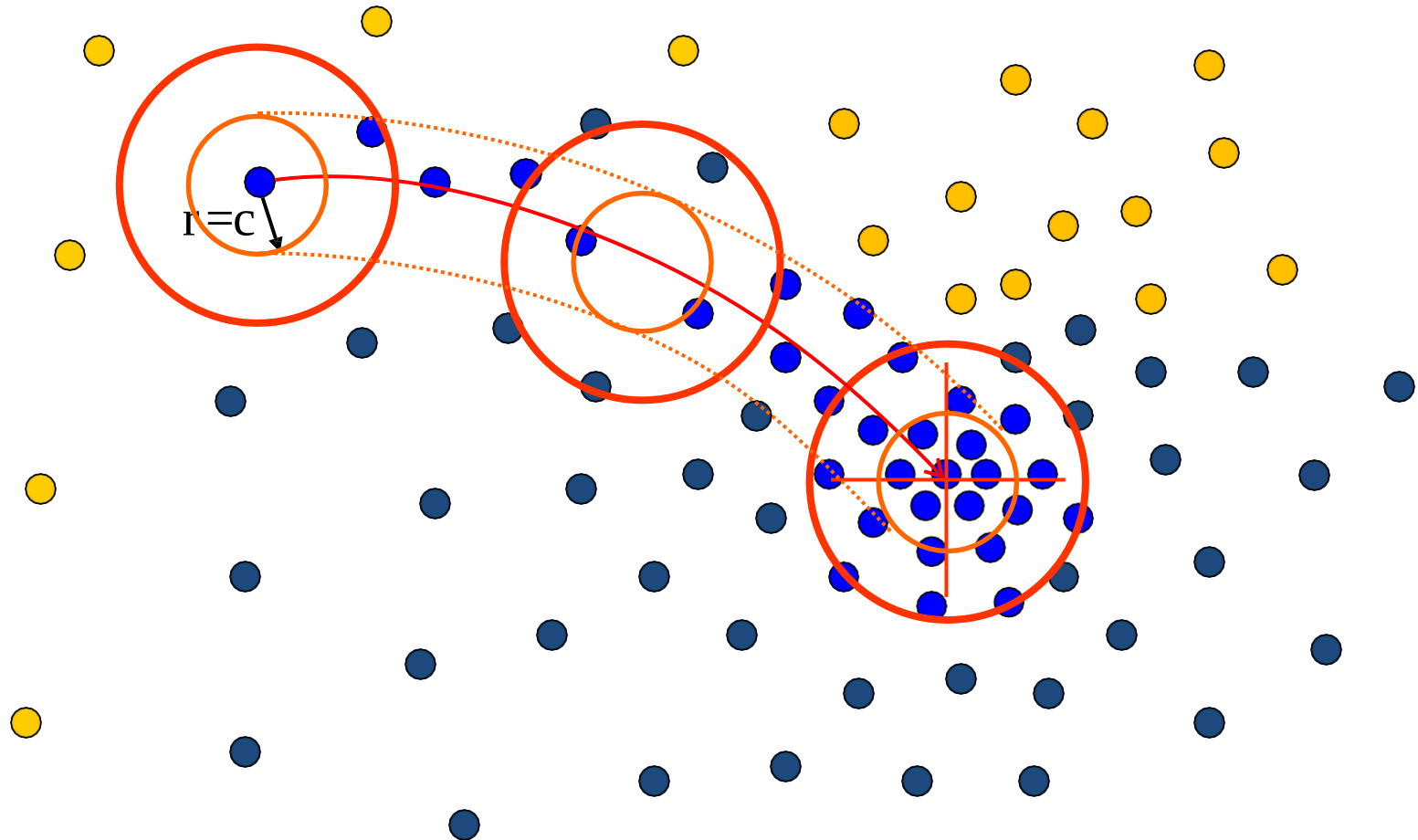
- Need to shift many windows...
- Many computations will be redundant.

Speedups: Basin of Attraction



- Assign all points within radius r of end point to the mode.

Speedups: Basin of Attraction



- Assign all points within radius r/c of the search path to the mode -> reduce the number of data points to search.

Technical Details

Given n data points $\mathbf{x}_i \in \mathbb{R}^d$, the multivariate kernel density estimate using a radially symmetric kernel¹ (e.g., Epanechnikov and Gaussian kernels), $K(\mathbf{x})$, is given by,

$$\hat{f}_K = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right), \quad (1)$$

where h (termed the *bandwidth* parameter) defines the radius of kernel. The radially symmetric kernel is defined as,

$$K(\mathbf{x}) = c_k k(\|\mathbf{x}\|^2), \quad (2)$$

where c_k represents a normalization constant.

Technical Details

A kernel is a function that satisfies the following requirements :

1. $\int_{\mathbb{R}^d} \phi(x) = 1$

2. $\phi(x) \geq 0$

Some examples of kernels include :

1. Rectangular $\phi(x) = \begin{cases} 1 & a \leq x \leq b \\ 0 & \text{else} \end{cases}$

2. Gaussian $\phi(x) = e^{-\frac{x^2}{2\sigma^2}}$

3. Epanechnikov $\phi(x) = \begin{cases} \frac{3}{4}(1 - x^2) & \text{if } |x| \leq 1 \\ 0 & \text{else} \end{cases}$

Technical Details

- Taking the derivative of: $\hat{f}_K = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)$

$$\nabla \hat{f}(\mathbf{x}) = \underbrace{\frac{2c_k}{nh^{d+2}} \left[\sum_{i=1}^n g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right) \right]}_{\text{term 1}} \underbrace{\left[\frac{\sum_{i=1}^n \mathbf{x}_i g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)}{\sum_{i=1}^n g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)} - \mathbf{x} \right]}_{\text{term 2}}, \quad (3)$$

where $g(x) = -k'(x)$ denotes the derivative of the selected kernel profile.

- Term1: this is proportional to the density estimate at $\mathbf{x}$ (similar to equation 1 from two slides ago).
- Term2: this is the mean-shift vector that points towards the direction of maximum density.

Technical Details

Finally, the mean shift procedure from a given point $\mathbf{x}$ is:

- Computer the mean shift vector $\mathbf{m}$:

$$\left[\frac{\sum_{i=1}^n \mathbf{x}_i g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)}{\sum_{i=1}^n g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)} - \mathbf{x} \right]$$

- Translate the density window:

$$\mathbf{x}^{t+1} = \mathbf{x}^t + \mathbf{m}(\mathbf{x}^t)$$

- Iterate steps 1 and 2 until convergence.

$$\nabla f(\mathbf{x}) = 0$$

Summary Mean-Shift

Pros

- General, application-independent tool
- Model-free, does not assume any prior shape (spherical, elliptical, etc.) on data clusters
- Just a single parameter (window size h)
 - h has a physical meaning (unlike k-means): Smoother density with larger h
- Finds variable number of modes
- Robust to outliers

Cons

- Output depends on window size
- Window size (bandwidth) selection is not trivial
- Computationally (relatively) expensive
- Does not scale well with dimension of feature space



What we have learned today?

- Introduction to segmentation and clustering
- Agglomerative clustering
- K-means clustering
- Mean-shift clustering
- Expectation-Maximization (EM)

Expectation-Maximization (EM)

- ❑ Additional latent variables allows to express relatively complex marginal distributions over latent variables in terms of more tractable joint distributions over the expanded space
- ❑ Maximum-Likelihood estimator in such a space is the Expectation-Maximization (EM) algorithm

K-means clustering: EM Optimization

1. Initialize ($t = 0$): cluster centers $c_1, \dots, c_K$

2. Compute δ^t : assign each point to the closest center

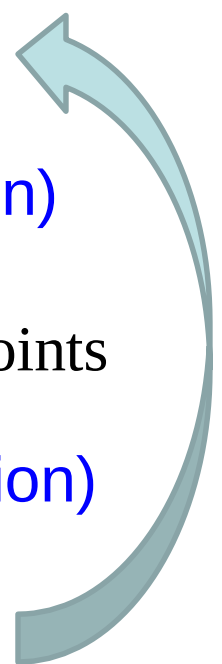
– δ^t denotes the set of assignment for each x_j to cluster C_i at iteration t

$$\delta^t = \underset{\delta}{\operatorname{argmin}} \frac{1}{N} \sum_j \sum_i^K \delta_{ij}^{t-1} (c_i^{t-1} - x_j)^2 \quad (\text{Expectation})$$

3. Computer c^t : update cluster centers as the mean of the points

$$c^t = \underset{c}{\operatorname{argmin}} \frac{1}{N} \sum_j \sum_i^K \delta_{ij}^t (c_i^{t-1} - x_j)^2 \quad (\text{Maximization})$$

4. Update $t = t + 1$, Repeat Step 2-3 till stopped



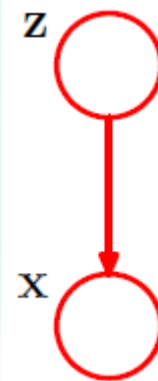


GMM: Latent variables

- Gaussian Mixture Distribution:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

- Latent variable $\mathbf{z}$
 - $\mathbf{z}$ is binary 1-of- K coding variable
 - $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})p(\mathbf{x}|\mathbf{z})$





GMM: Latent variables

- $p(z_k = 1) = \pi_k$
constraints: $0 \leq \pi_k \leq 1$, and $\sum_k \pi_k = 1$
 $p(\mathbf{z}) = \prod_k \pi_k^{z_k}$
- $p(\mathbf{x}|z_k = 1) = \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$
 $p(\mathbf{x}|\mathbf{z}) = \prod_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)^{z_k}$
- $p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}) = \sum_{\mathbf{z}} p(\mathbf{z})p(\mathbf{x}|\mathbf{z}) = \sum_k \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$
- The use of the joint probability $p(\mathbf{x}, \mathbf{z})$, leads to significant simplifications optimized by EM



GMM: Latent variables

- Posterior probability:

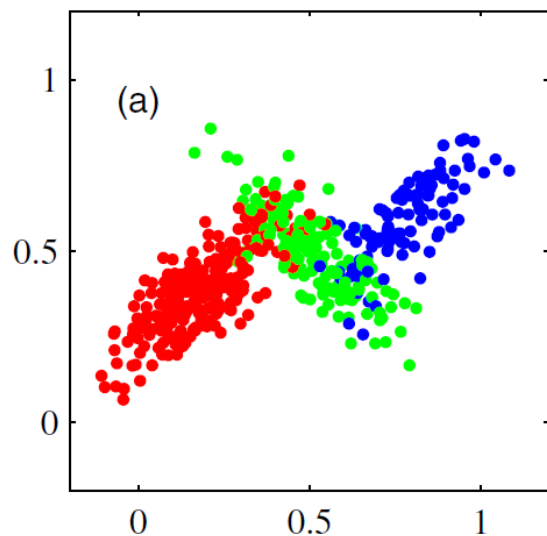
$$\begin{aligned}\gamma(z_k) \equiv p(z_k = 1 | \mathbf{x}) &= \frac{p(z_k = 1)p(\mathbf{x} | z_k = 1)}{\sum_k p(z_k = 1)p(\mathbf{x} | z_k = 1)} \\ &= \frac{\pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)}{\sum_k \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)}\end{aligned}$$

- Can be interpreted as responsibility of component k to explain observation $\mathbf{x}$

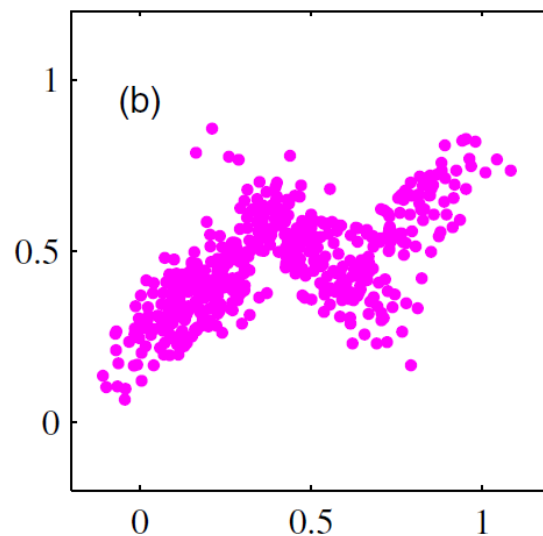


GMM: Latent variables

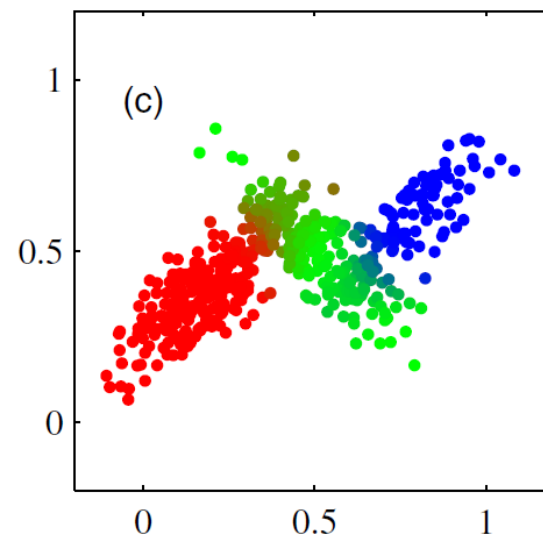
- Generate random samples with given parameters:
 1. Generate $\hat{\mathbf{z}}$ from $p(\mathbf{z})$
 2. Generate $\hat{\mathbf{x}}$ from $p(\mathbf{x}|\hat{\mathbf{z}})$



(a) $p(\mathbf{z})p(\mathbf{x}|\mathbf{z})$



(b) $p(\mathbf{x})$, missing $\mathbf{z}$



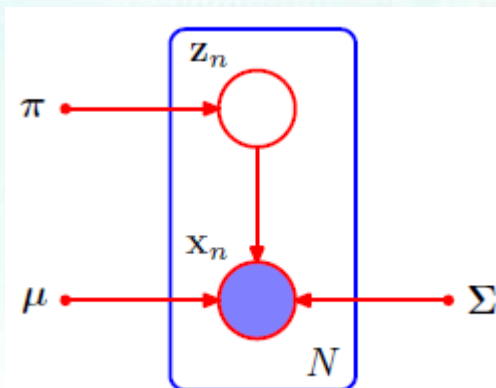
(c) Estimated $\mathbf{z}$



GMM: Maximum Likelihood

- Log Likelihood

$$\ln p(\mathbf{X}|\pi, \mu, \Sigma) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k) \right\}$$





GMM: Optimization

- Maximum of log likelihood:
 - Set derivatives of $\ln p(\mathbf{X}|\pi, \mu, \Sigma)$ w.r.t parameters to 0
- For the μ_k :

$$0 = - \sum_{n=1}^N \underbrace{\frac{\pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}{\sum_k \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}}_{\gamma(z_k)} \Sigma_k^{-1} (\mathbf{x}_n - \mu_k)$$

$$\mu_k = \frac{1}{\sum_n \gamma(z_k)} \sum_n \gamma(z_k) \mathbf{x}_n$$



GMM: Optimization

- For the Σ_k :

$$\Sigma_k = \frac{1}{\sum_n \gamma(z_k)} \sum_n \gamma(z_k) (\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^T$$

- For the π_k :

- Take into account constraint $\sum_k \pi_k$

- Lagrange multiplier

$$\ln p(\mathbf{X}|\pi, \mu, \Sigma) + \lambda(\sum_k \pi_k - 1)$$

$$0 = \sum_n \frac{\mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}{\sum_k \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)} + \lambda$$

$$\pi_k = \frac{\sum_n \gamma(z_k)}{N}$$

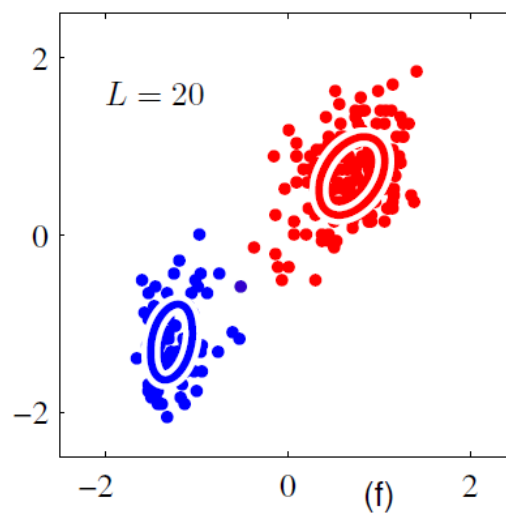
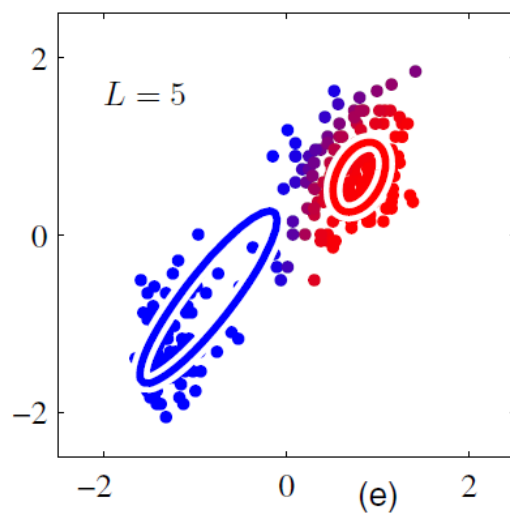
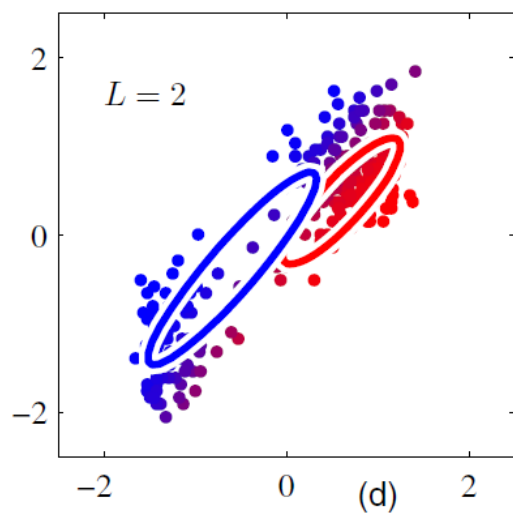
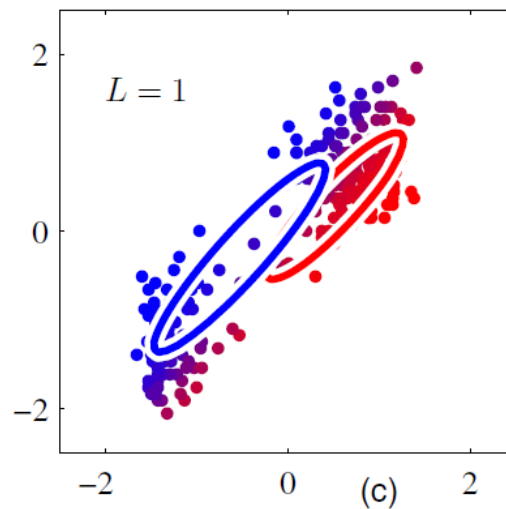
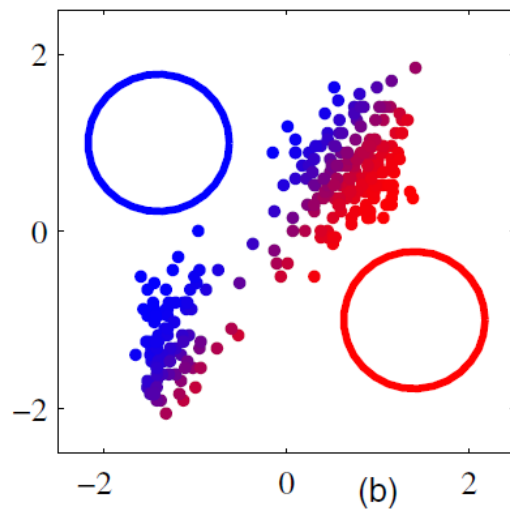
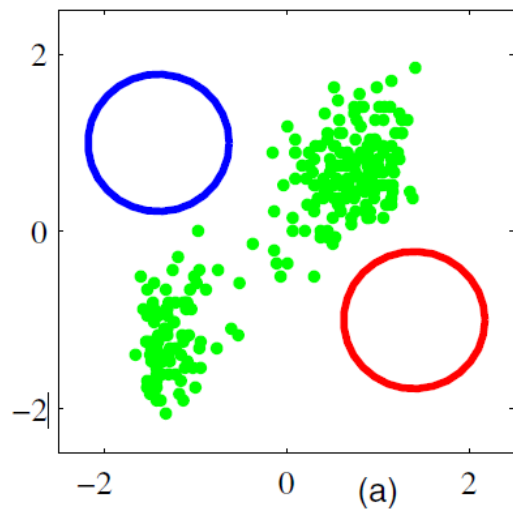


GMM: EM Optimization

- No closed form solutions:
 - $\gamma(z_k)$ depends on parameters
- Iterative scheme for finding maximum likelihood:
 - Alternate between estimating the current $\gamma(z_k)$ and updating the parameters $\{\pi_k, \mu_k, \Sigma_k\}$
- Notes:
 - More iterations needed to converge than K-means algorithm, and each cycle requires more computation
 - Common, initialize parameters based K-means



GMM: Illustration





GMM: Summary

1. Initialize $\{\mu_k, \Sigma_k, \pi_k\}$ and evaluate log-likelihood
2. **E-Step** Evaluate responsibilities $\gamma(z_k)$
3. **M-Step** Re-estimate parameters, using current responsibilities:

$$\mu_k^{\text{new}} = \frac{1}{\sum_n \gamma(z_k)} \sum_n \gamma(z_k) \mathbf{x}_n$$

$$\Sigma_k^{\text{new}} = \frac{1}{\sum_n \gamma(z_k)} \sum_n \gamma(z_k) (\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^T$$

$$\pi_k^{\text{new}} = \frac{\sum_n \gamma(z_k)}{N}$$

4. Evaluate log-likelihood $\ln p(\mathbf{X}|\pi, \mu, \Sigma)$ and check for convergence (go to step 2).



Alternative View of EM

- ▶ Let $\mathbf{X}$ observed data, $\mathbf{Z}$ latent variables, θ parameters.
- ▶ Goal: maximize marginal log-likelihood of observed data

$$\ln p(\mathbf{X}|\theta) = \ln \left\{ \sum_{\mathbf{Z}} p(\mathbf{X}, \mathbf{Z}|\theta) \right\}.$$

- ▶ Optimization problematic due to log-sum.
- ▶ Assume straightforward maximization for complete data

$$\ln p(\mathbf{X}, \mathbf{Z}|\theta).$$

- ▶ Latent $\mathbf{Z}$ is known only through $p(\mathbf{Z}|\mathbf{X}, \theta)$.
- ▶ We will consider expectation of complete data log-likelihood.



Alternative View of EM

- ▶ **Initialization:** Choose initial set of parameters θ^{old} .
- ▶ **E-step:** use current parameters θ^{old} to compute $p(\mathbf{Z}|\mathbf{X}, \theta^{old})$ to find expected complete-data log-likelihood for general θ

$$Q(\theta, \theta^{old}) = \sum_{\mathbf{Z}} p(\mathbf{Z}|\mathbf{X}, \theta^{old}) \ln p(\mathbf{X}, \mathbf{Z}|\theta).$$

- ▶ **M-step:** determine θ^{new} by maximizing $Q(\theta, \theta^{old})$

$$\theta^{new} = \arg \max_{\theta} Q(\theta, \theta^{old}).$$

- ▶ **Check convergence:** stop, or $\theta^{old} \leftarrow \theta^{new}$ and go to **E-step**.



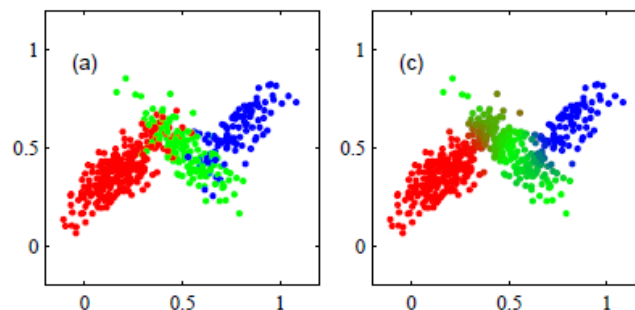
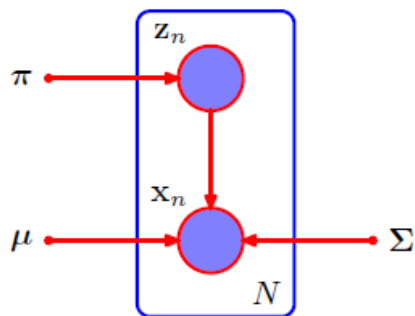
► For mixture assign each $\mathbf{x}$ latent **assignment variables** z_k .

► Complete-data (log-)likelihood and expectation

$$p(\mathbf{x}, \mathbf{z} | \theta) = \prod_{k=1}^K \pi_k^{z_k} \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_k}$$

$$\ln p(\mathbf{x}, \mathbf{z} | \theta) = \sum_{k=1}^K z_k \{ \ln \pi_k + \ln \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \}$$

$$\mathcal{Q}(\theta) = \mathbb{E}_{\mathbf{z}}[\ln p(\mathbf{x}, \mathbf{z} | \theta)] = \sum_{k=1}^K \gamma(z_k) \{ \ln \pi_k + \ln \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \}$$





EM Algorithm in General

- ▶ Let $\mathbf{X}$ observed data, $\mathbf{Z}$ latent variables, θ parameters.
- ▶ Goal: maximize marginal log-likelihood of observed data

$$\ln p(\mathbf{X}|\theta) = \ln \left\{ \sum_{\mathbf{Z}} p(\mathbf{X}, \mathbf{Z}|\theta) \right\}.$$

- ▶ Maximization of $p(\mathbf{X}, \mathbf{Z}|\theta)$ simple, but difficult for $p(\mathbf{X}|\theta)$.
- ▶ Given any $q(\mathbf{Z})$, we decompose the data log-likelihood

$$\ln p(\mathbf{X}|\theta) = \mathcal{L}(q, \theta) + \text{KL}(q(\mathbf{Z})||p(\mathbf{Z}|\mathbf{X}, \theta)),$$

$$\mathcal{L}(q, \theta) = \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \frac{p(\mathbf{X}, \mathbf{Z}|\theta)}{q(\mathbf{Z})},$$

$$\text{KL}(q(\mathbf{Z})||p(\mathbf{Z}|\mathbf{X}, \theta)) = - \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \frac{p(\mathbf{Z}|\mathbf{X}, \theta)}{q(\mathbf{Z})} \geq 0.$$



► $\mathcal{L}(q, \theta)$ is a **lower bound on the data log-likelihood**

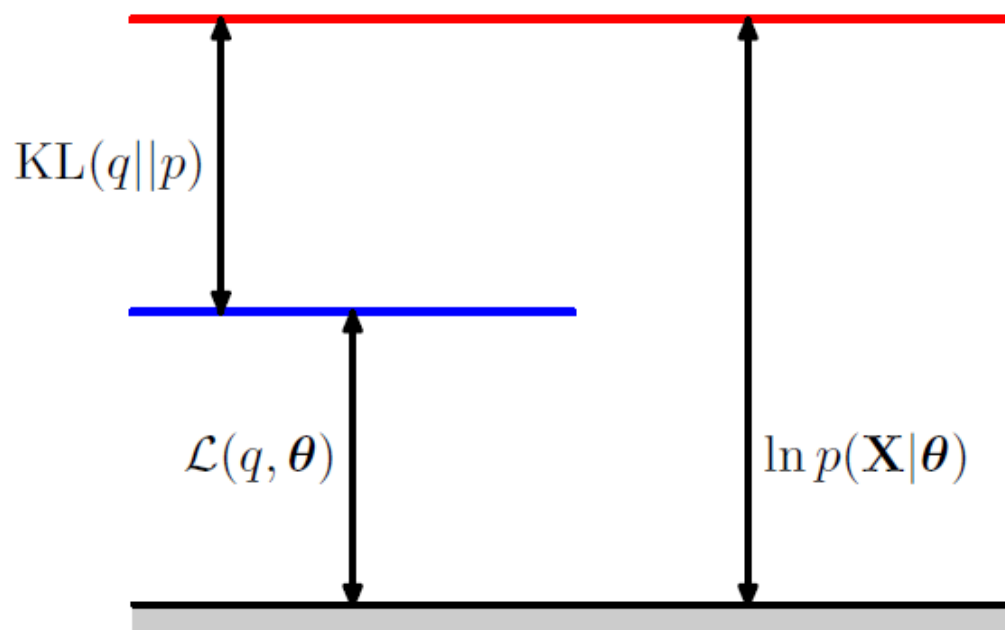
► $-\mathcal{L}(q, \theta)$ known as variational free-energy

$$\mathcal{L}(q, \theta) = \ln p(\mathbf{X}|\theta) - \text{KL}(q(\mathbf{Z})||p(\mathbf{Z}|\mathbf{X}, \theta)) \leq \ln p(\mathbf{X}|\theta)$$

► **The EM algorithm performs coordinate ascent on $\mathcal{L}$**

► E-step maximizes $\mathcal{L}$ w.r.t. q for fixed θ

► M-step maximizes $\mathcal{L}$ w.r.t. θ for fixed q



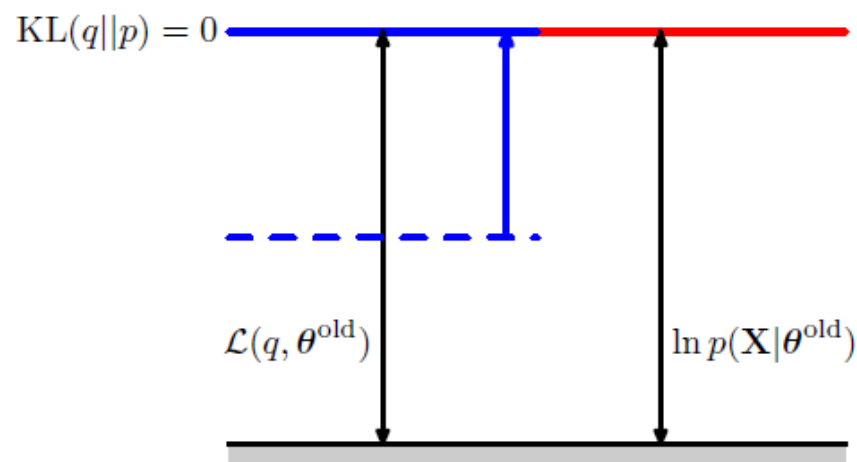
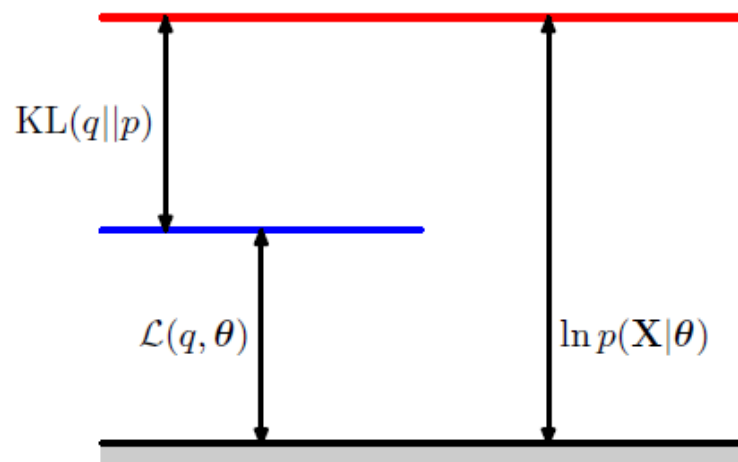


E-step

- ▶ E-step maximizes $\mathcal{L}(q, \theta)$ w.r.t. q for fixed θ

$$\mathcal{L}(q, \theta) = \ln p(\mathbf{X}|\theta) - \text{KL}(q(\mathbf{Z})||p(\mathbf{Z}|\mathbf{X}, \theta))$$

- ▶ $\mathcal{L}$ maximized for $q(\mathbf{Z}) \leftarrow p(\mathbf{Z}|\mathbf{X}, \theta)$



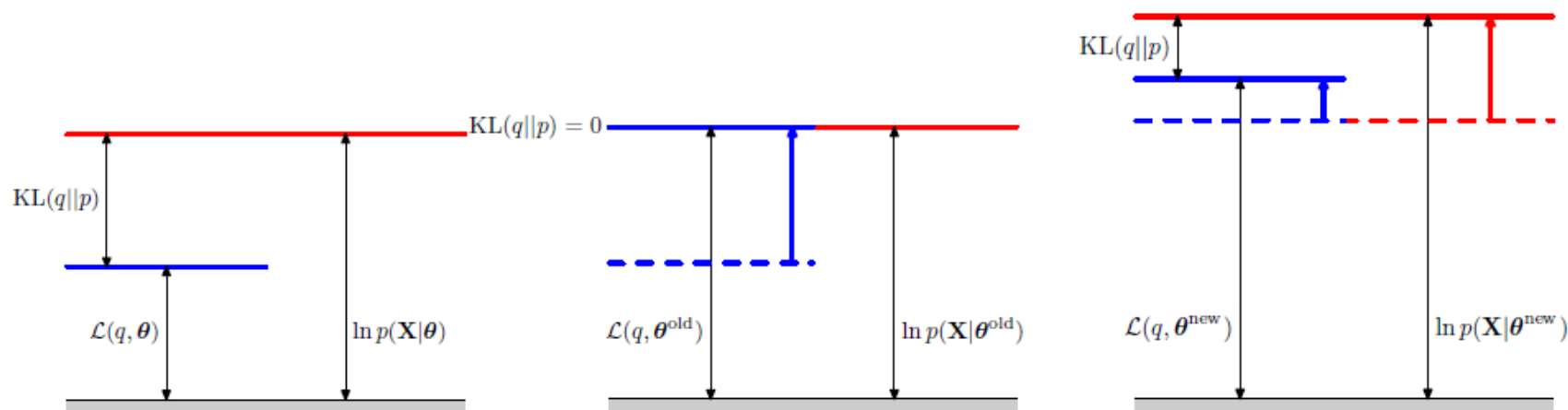


M-step

- M-step maximizes $\mathcal{L}(q, \theta)$ w.r.t. θ for fixed q

$$\mathcal{L}(q, \theta) = \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln p(\mathbf{X}, \mathbf{Z} | \theta) - \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln q(\mathbf{Z})$$

- $\mathcal{L}$ maximized for $\theta = \arg \max_{\theta} \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln p(\mathbf{X}, \mathbf{Z} | \theta)$



EM Algorithm: Final Thoughts

- ▶ (local) maxima of $\mathcal{L}(q, \theta)$ correspond to those of $\ln p(\mathbf{X}|\theta)$
- ▶ EM converges to (local) maximum of likelihood
 - ▶ Coordinate ascent on $\mathcal{L}(q, \theta)$, and $\mathcal{L} = \ln p(\mathbf{X}|\theta)$ after E-step
- ▶ Alternative schemes to optimize the bound
 - ▶ Generalized EM: relax M-step from maximizing to increasing $\mathcal{L}$
 - ▶ Expectation Conditional Maximization: M-step maximizes w.r.t. groups of parameters in turn
 - ▶ Incremental EM: E-step per data point, incremental M-step
 - ▶ Variational EM: relax E-step from maximizing to increasing $\mathcal{L}$
 - ▶ no longer $\mathcal{L} = \ln p(\mathbf{X}|\theta)$ after E-step
- ▶ Same applies for MAP estimation $p(\theta|\mathbf{X}) = p(\theta)p(\mathbf{X}|\theta)/p(\mathbf{X})$
 - ▶ bound second term: $\ln p(\theta|\mathbf{X}) \geq \ln p(\theta) + \mathcal{L}(q, \theta) - \ln p(\mathbf{X})$

What we have learned today?

- Introduction to segmentation and clustering
- Agglomerative clustering
- K-means clustering
- Mean-shift clustering
- Expectation-Maximization (EM)